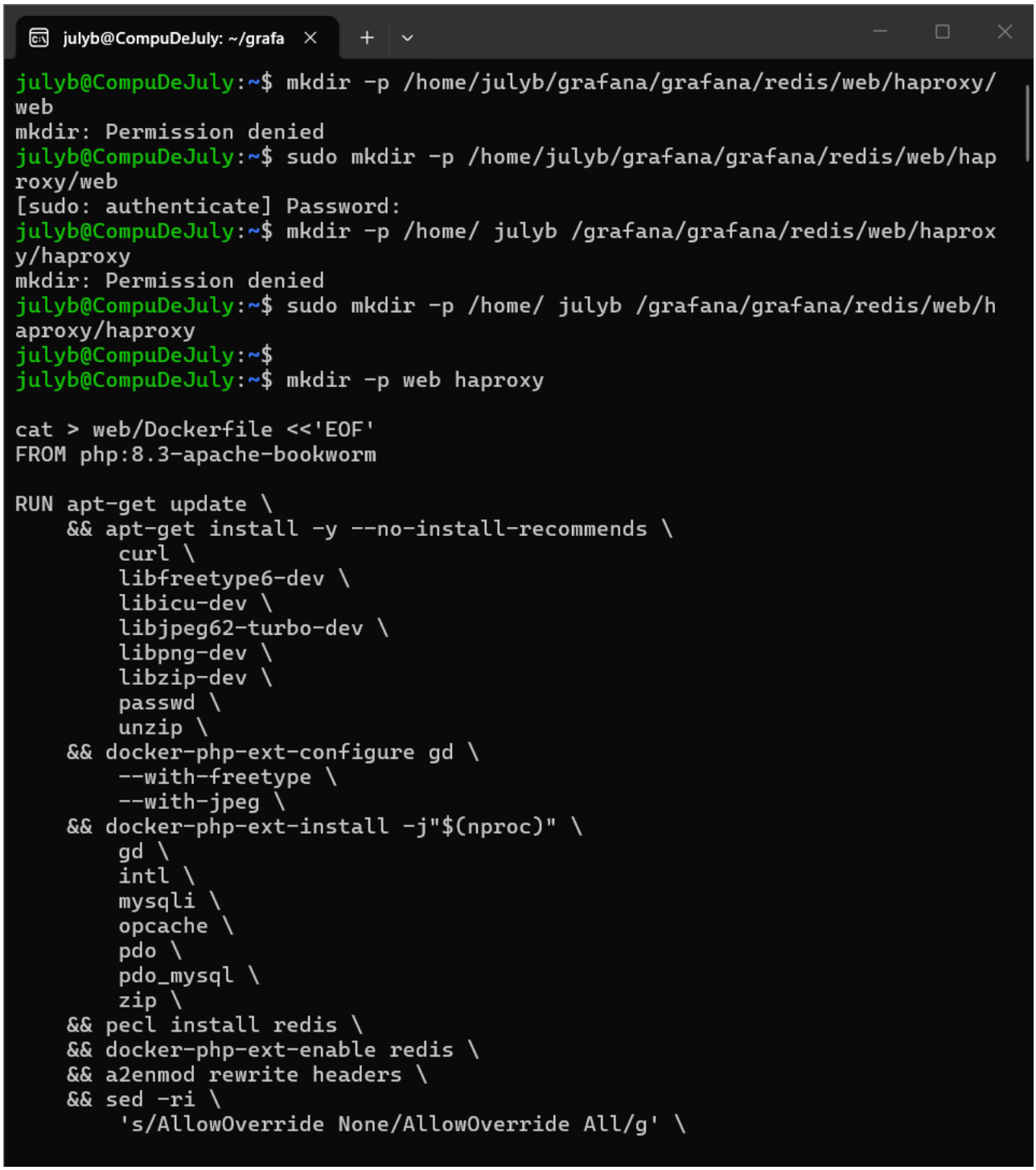
Creación del usuario orh en el host con UID/GID 1003 y asignación al grupo docker.



Corrección de permisos de carpetas con sudo e inicio de la escritura del Dockerfile.

julyb@CompuDeJuly: ~/grafa

```
    && sed -ri \
        's/AllowOverride None/AllowOverride All/g' \
        /etc/apache2/apache2.conf \
    && echo "ServerName localhost" \
> /etc/apache2/conf-available/servername.conf \
    && a2enconf servername \
    && rm -rf /var/lib/apt/lists/*

RUN printf '\n# Permisos predeterminados de la aplicación\numask 0027\n' \
    >> /etc/apache2/envvars

COPY redis-session.ini \
    /usr/local/etc/php/conf.d/zz-redis-session.ini

COPY docker-entrypoint.sh \
    /usr/local/bin/wordpress-entrypoint

RUN chmod 0755 /usr/local/bin/wordpress-entrypoint

ENTRYPOINT ["wordpress-entrypoint"]
CMD ["apache2-foreground"]
EOF

cat > web/redis-session.ini <<'EOF'
; WordPress Core no utiliza normalmente sesiones PHP para autenticar.
; Esta configuración cubre plugins que sí llaman session_start().

session.save_handler = redis
session.save_path = "tcp://redis:6379?database=1&prefix=PHPREDIS_SESSION:"

session.use_strict_mode = 1
session.cookie_httponly = 1
session.cookie_samesite = "Lax"
EOF

cat > web/docker-entrypoint.sh <<'EOF'
#!/usr/bin/env bash
set -Eeuo pipefail

APP_UID="${APP_UID:-33}"
APP_GID="${APP_GID:-33}"

error() {
```

Continuación del Dockerfile (configuración de Apache, extensiones PHP) y creación de redis-session.ini.

julyb@CompuDeJuly: ~/grafa

```
error() {
    printf 'ERROR: %s\n' "$*" >&2
    exit 1
}

case "${APP_UID}" in
    ''|*[!0-9]*)
        error "APP_UID debe ser un entero positivo."
        ;;
esac

case "${APP_GID}" in
    ''|*[!0-9]*)
        error "APP_GID debe ser un entero positivo."
        ;;
esac

if [ "$(id -u)" -ne 0 ]; then
    error "El contenedor debe iniciar como root para ajustar www-data."
fi

if ! id www-data >/dev/null 2>&1; then
    error "No existe el usuario interno www-data."
fi

current_uid="$(id -u www-data)"
current_gid="$(id -g www-data)"

uid_owner="$(
    getent passwd "${APP_UID}" |
    cut -d: -f1 ||
    true
)"

gid_owner="$(
    getent group "${APP_GID}" |
    cut -d: -f1 ||
    true
)"

if [ -n "${uid_owner}" ] && [ "${uid_owner}" != "www-data" ]; then
    error "El UID ${APP_UID} ya pertenece a ${uid_owner} dentro del contened
or."
```

Escritura del script docker-entrypoint.sh para la validación inicial de variables y usuarios.

julyb@CompuDeJuly: ~/grafa

```
or."
fi

if [ -n "${gid_owner}" ] && [ "${gid_owner}" != "www-data" ]; then
    error "El GID ${APP_GID} ya pertenece al grupo ${gid_owner} dentro del c
ontenedor."
fi

if [ "${current_gid}" != "${APP_GID}" ]; then
    groupmod --gid "${APP_GID}" www-data
fi

if [ "${current_uid}" != "${APP_UID}" ] || \
   [ "${current_gid}" != "${APP_GID}" ]; then
    usermod \
        --uid "${APP_UID}" \
        --gid "${APP_GID}" \
        www-data
fi

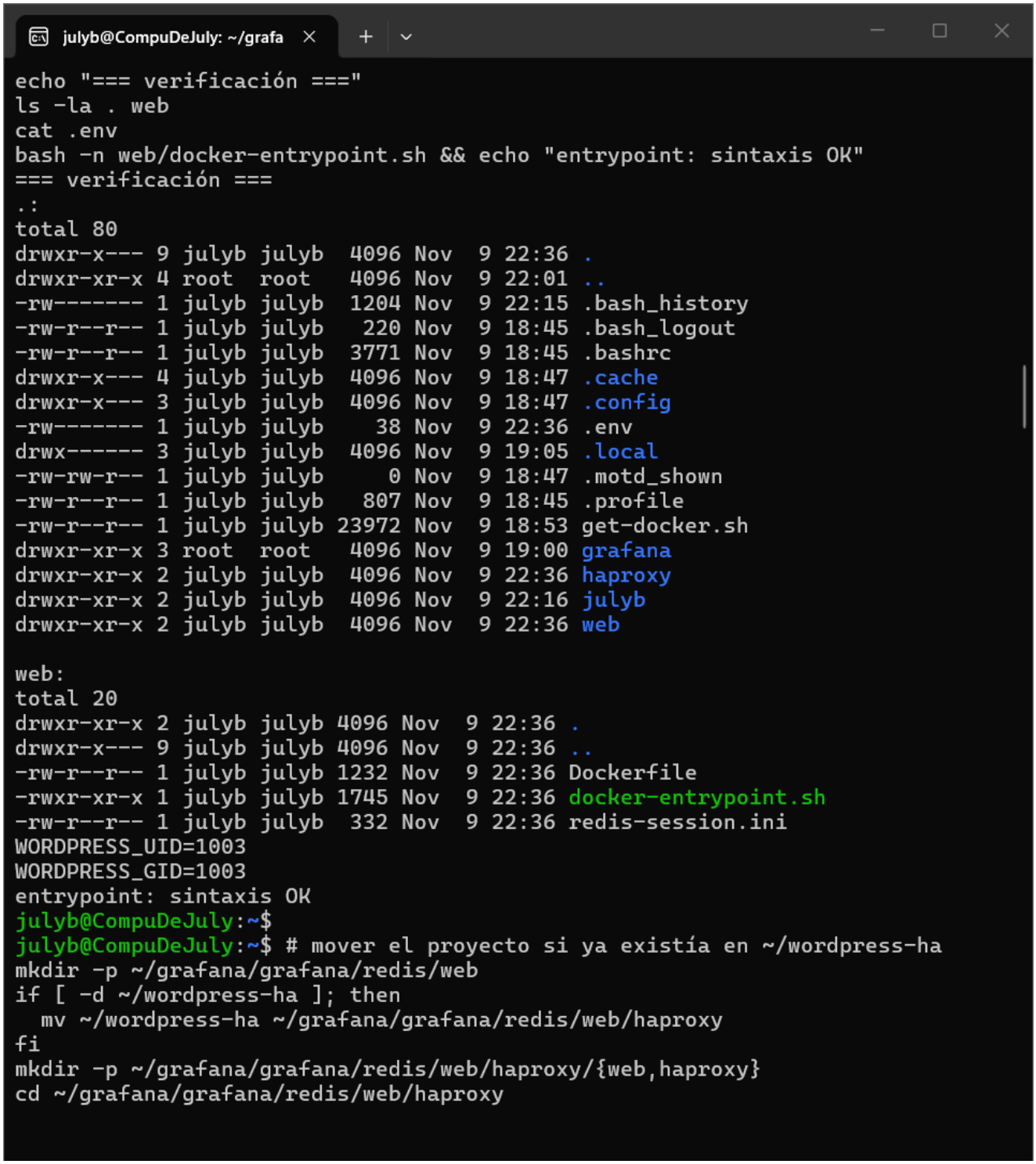
# Solo se preparan directorios internos de ejecución.
# No se cambia recursivamente la propiedad de /var/www/html.
install -d \
    -o www-data \
    -g www-data \
    -m 0755 \
    /var/run/apache2 \
    /var/lock/apache2

printf \
    'Iniciando Apache: www-data usa UID=%s GID=%s\n' \
    "$(id -u www-data)" \
    "$(id -g www-data)"

exec docker-php-entrypoint "$@"
EOF
chmod 0755 web/docker-entrypoint.sh

cat > .env <<'EOF'
WORDPRESS_UID=1003
WORDPRESS_GID=1003
EOF
chmod 600 .env
```

Finalización del entrypoint (ajuste dinámico de www-data al UID 1003) y creación del archivo .env.



Verificación de sintaxis del entrypoint y movimiento de carpetas del proyecto.



Inicio de la escritura del compose.yaml (configuración de MariaDB).



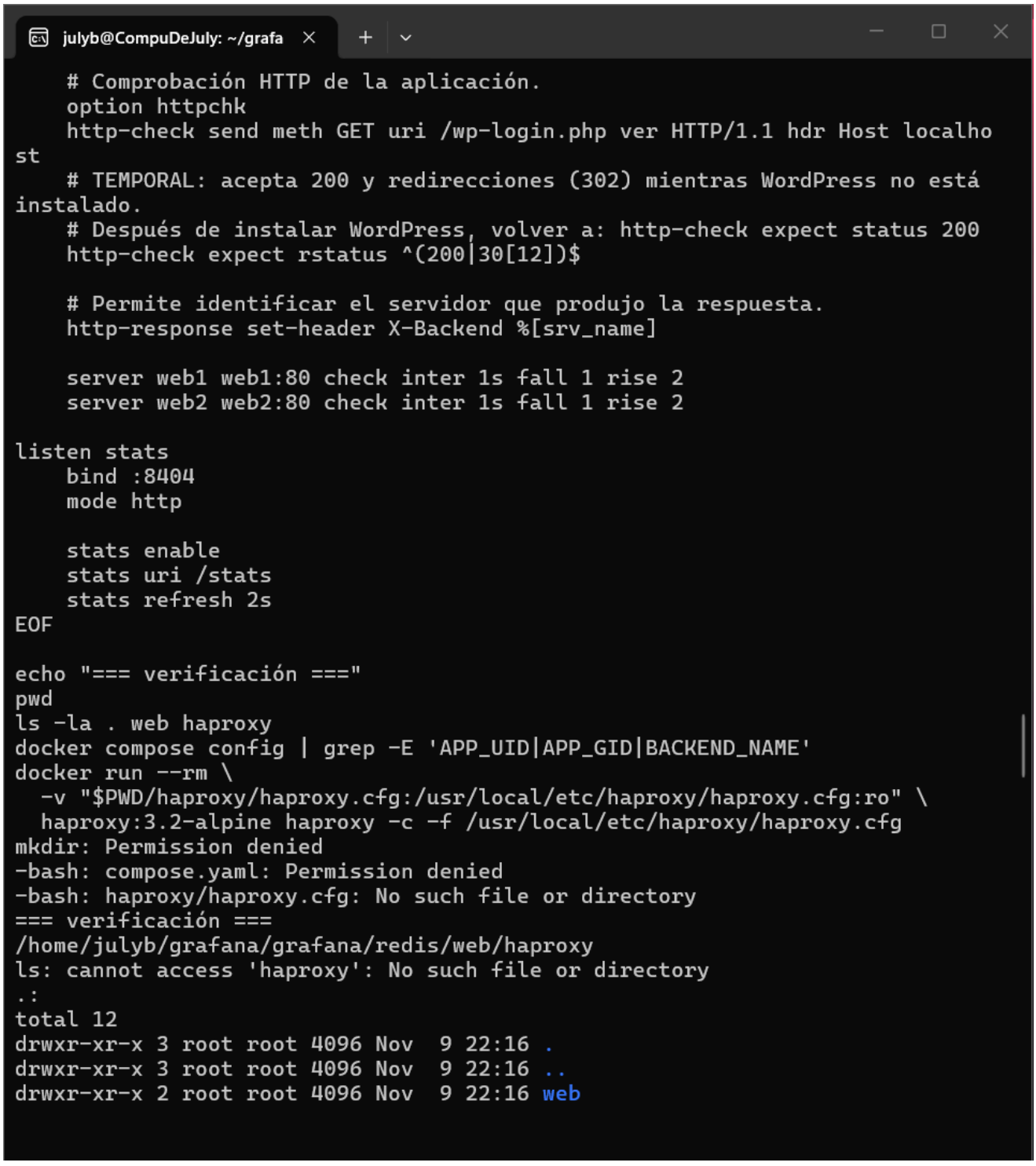
Configuración de Redis y healthchecks en el compose.yaml, e inicio del servicio web1.



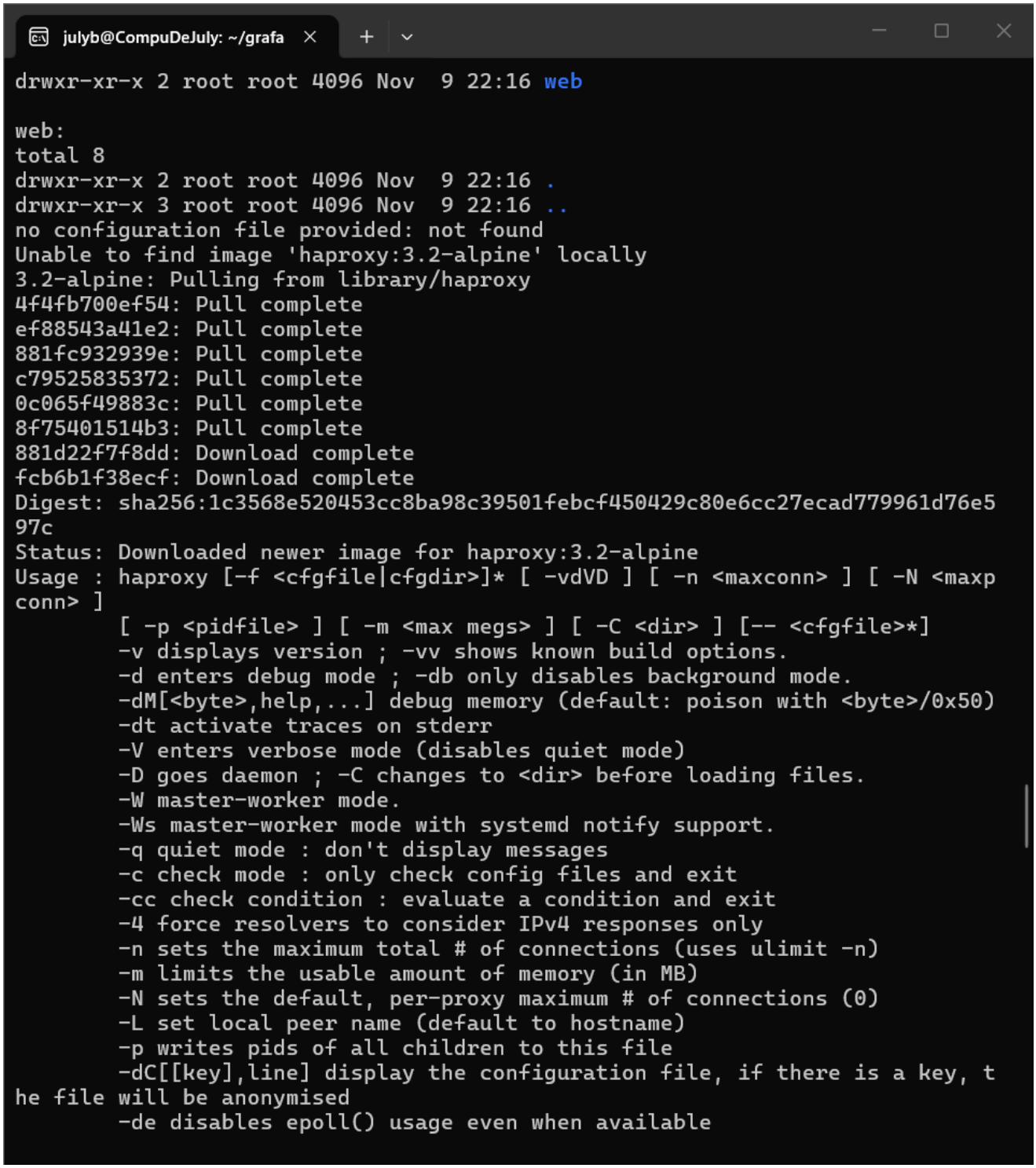
Configuración de web2 y HAProxy (puertos y volúmenes) en el compose.yaml.



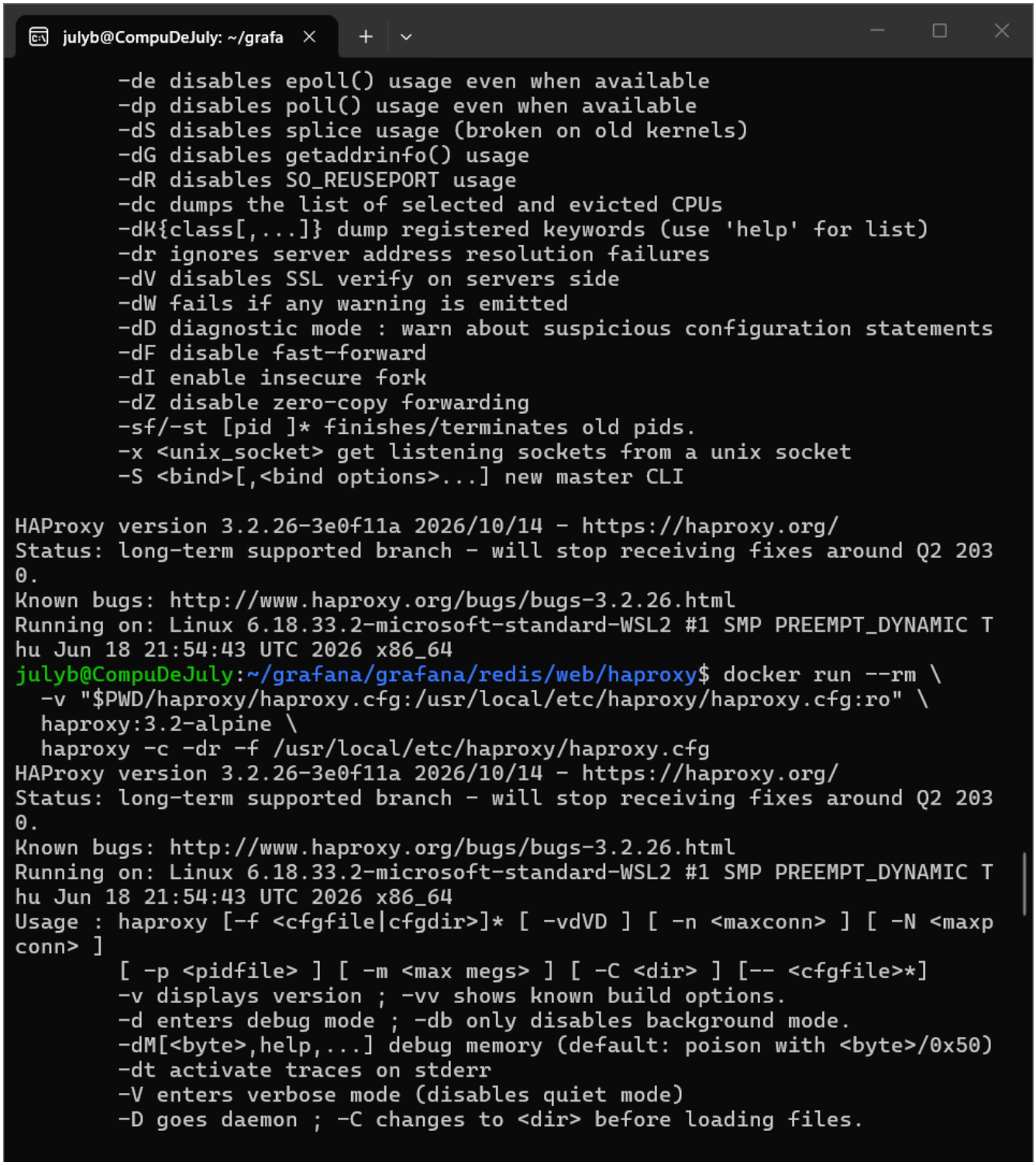
Cierre del compose.yaml y comienzo de la configuración de HAProxy (haproxy.cfg).



Configuración de balanceo, chequeos (fall 1, rise 2) y panel de estadísticas en HAProxy; error de validación por permisos.



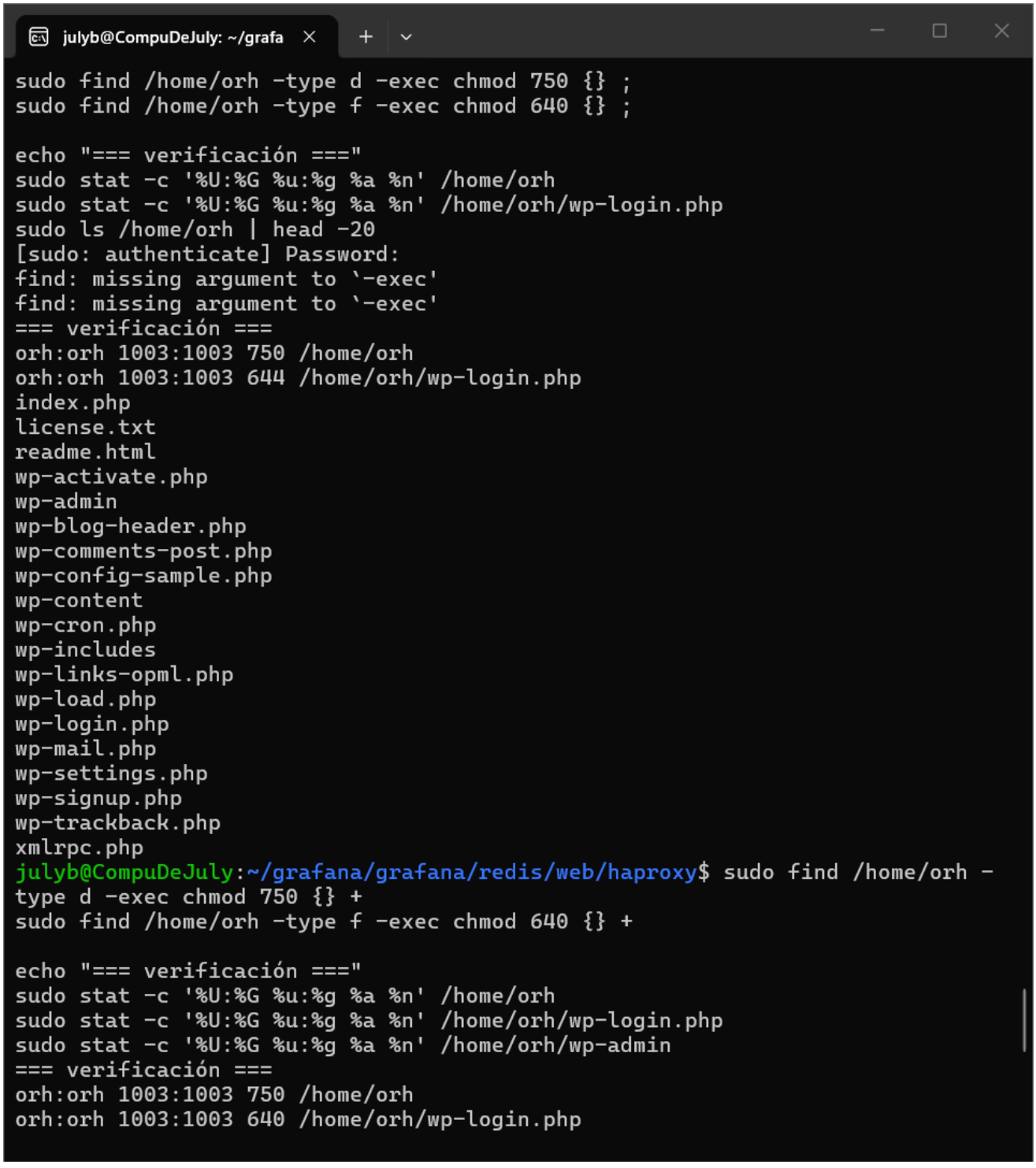
Intento de validar la configuración de HAProxy (error tipográfico) y descarga de la imagen oficial.



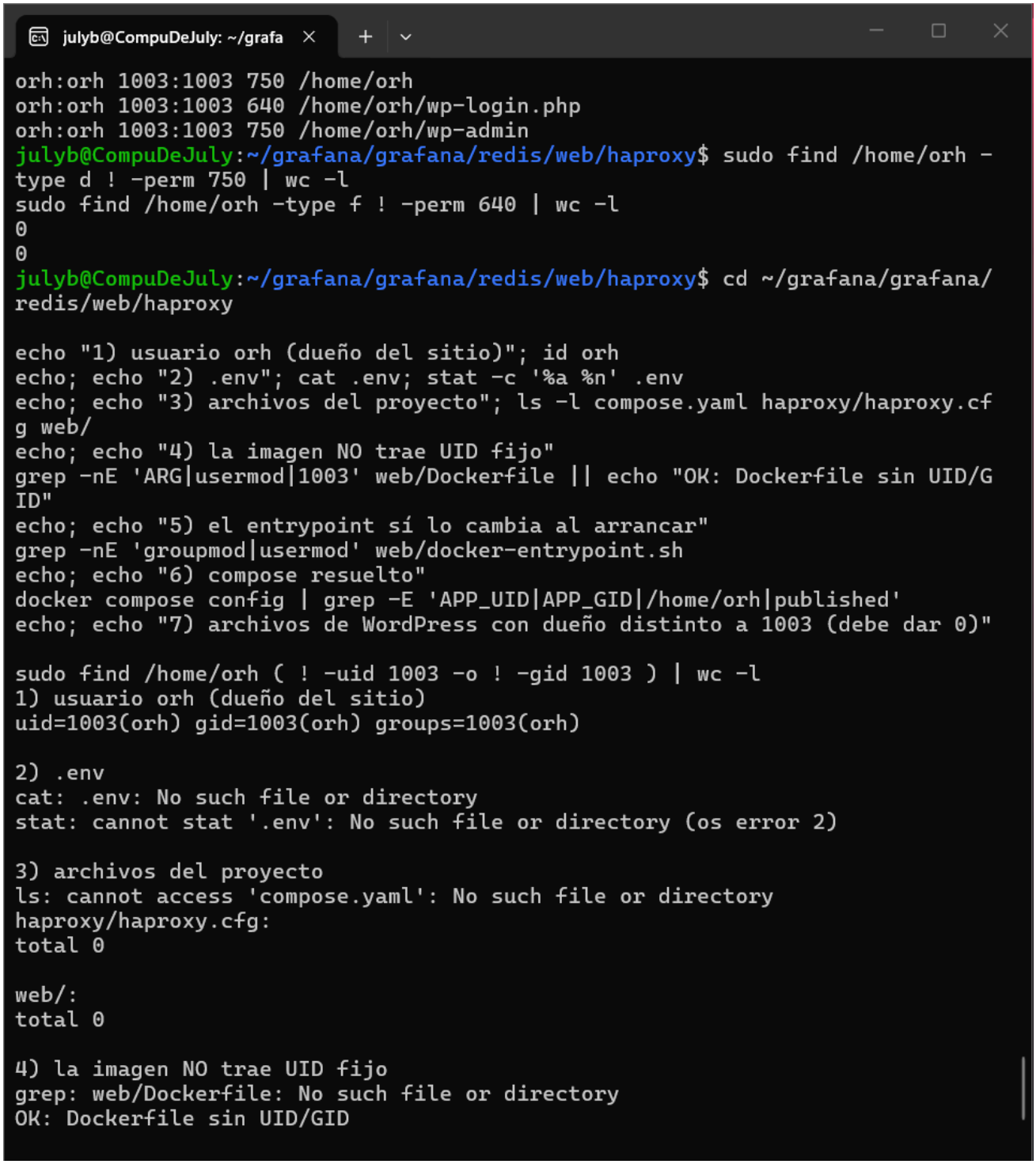
Salida de ayuda de HAProxy e inicio de la descarga del código fuente de WordPress.



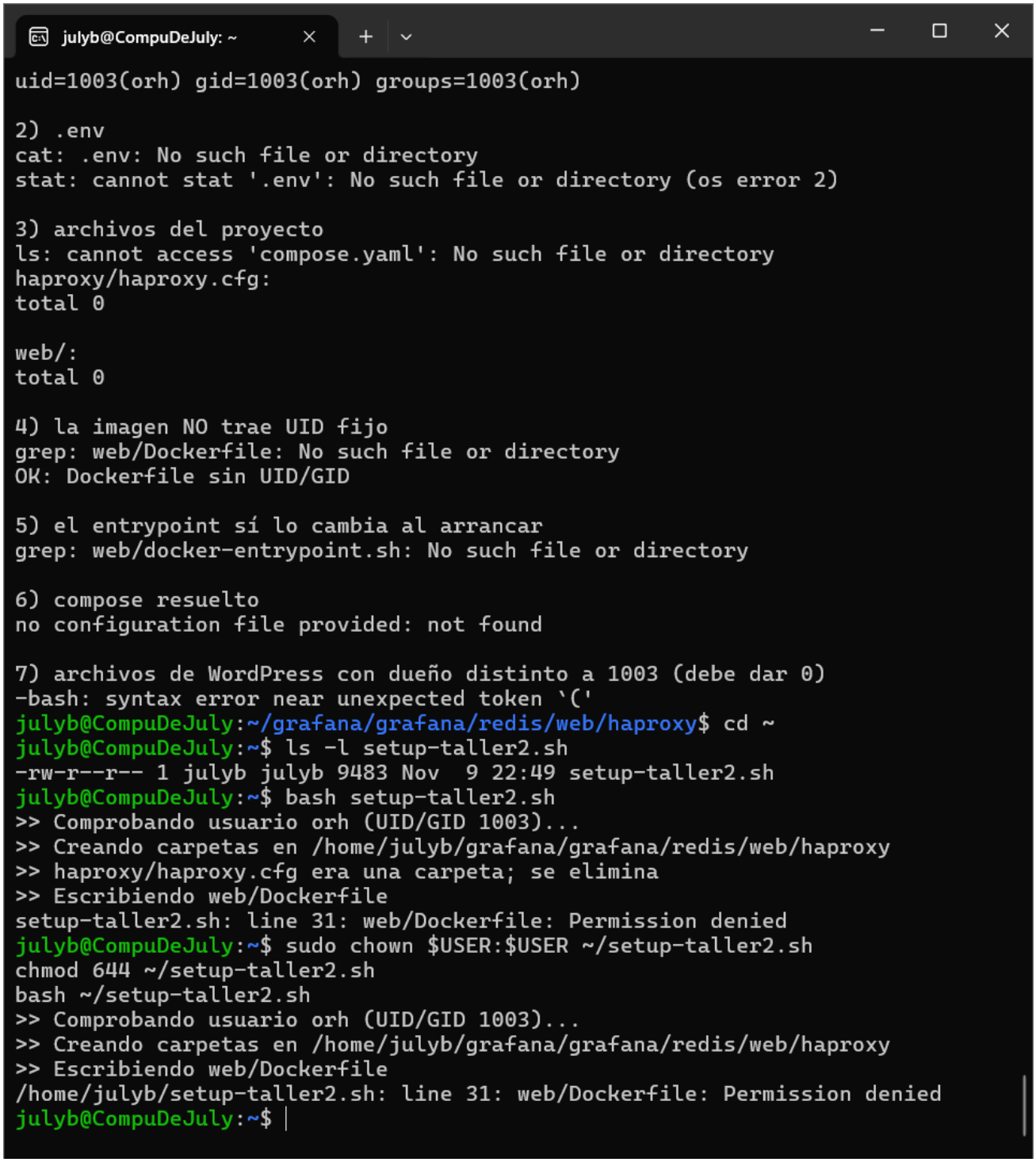
Finalización de la salida de ayuda y listado de los archivos de WordPress descargados.



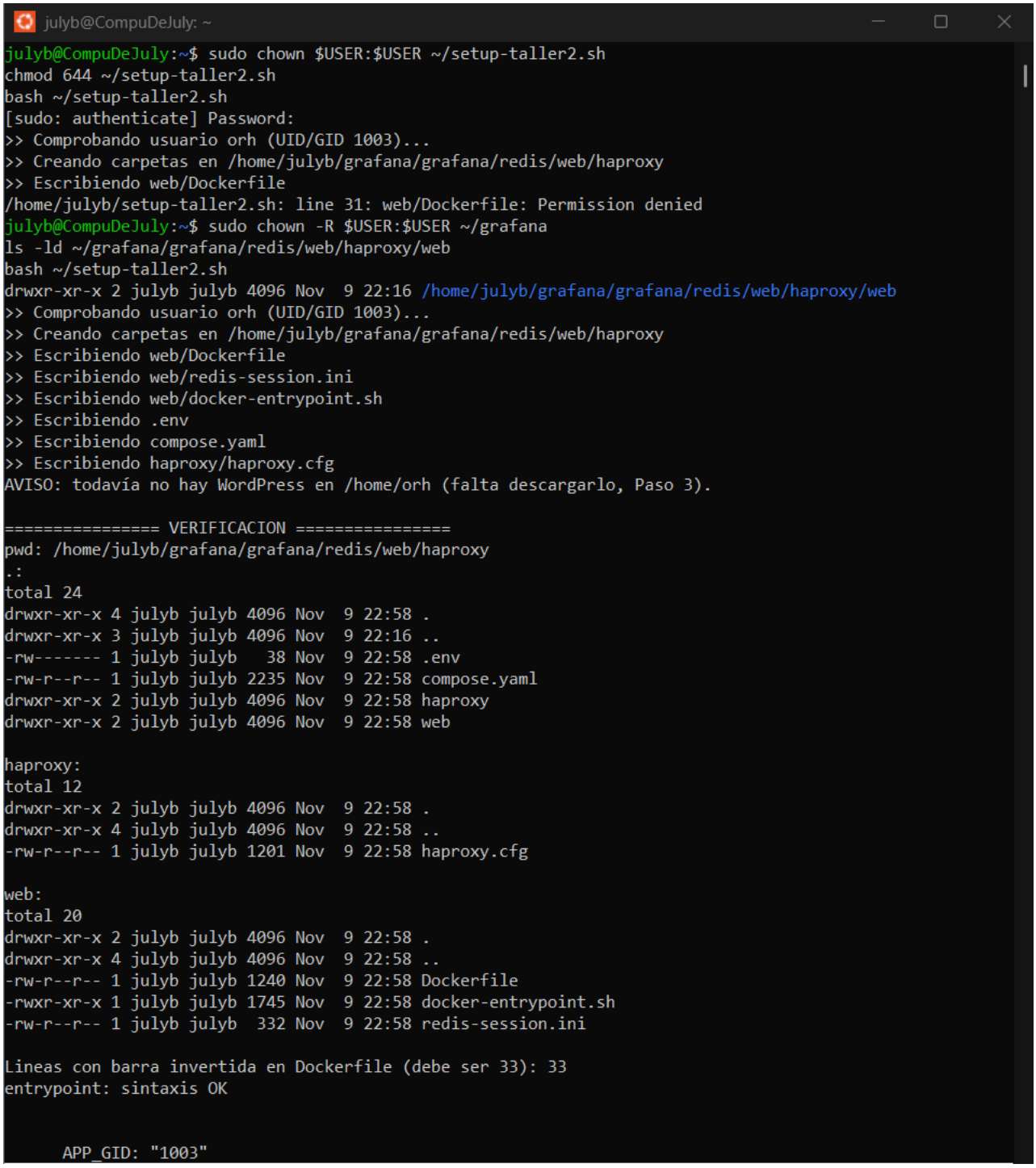
Aplicación de permisos restrictivos (750 para carpetas, 640 para archivos) en /home/orh.



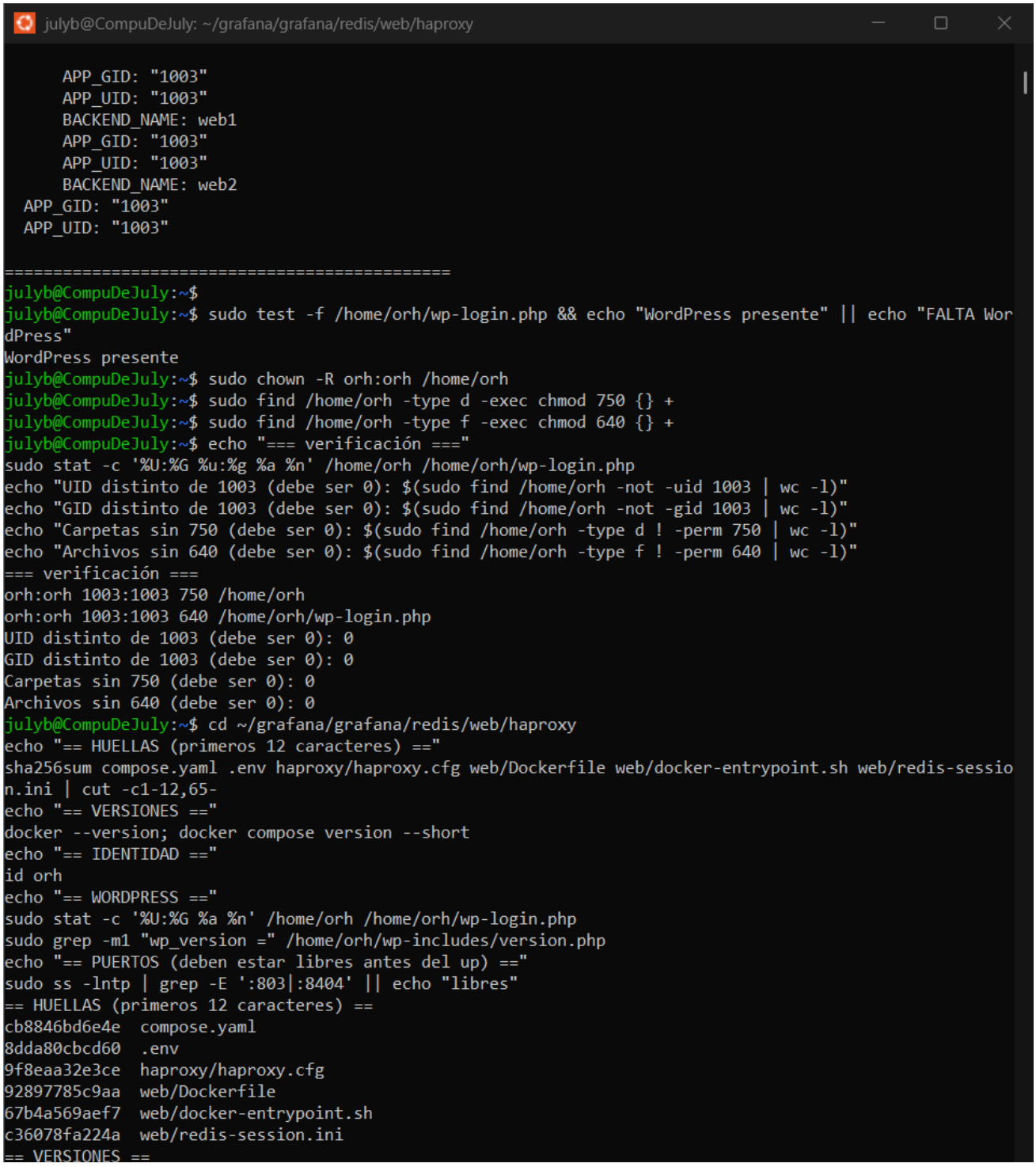
- Verificación de que los permisos en /home/orh se aplicaron correctamente.



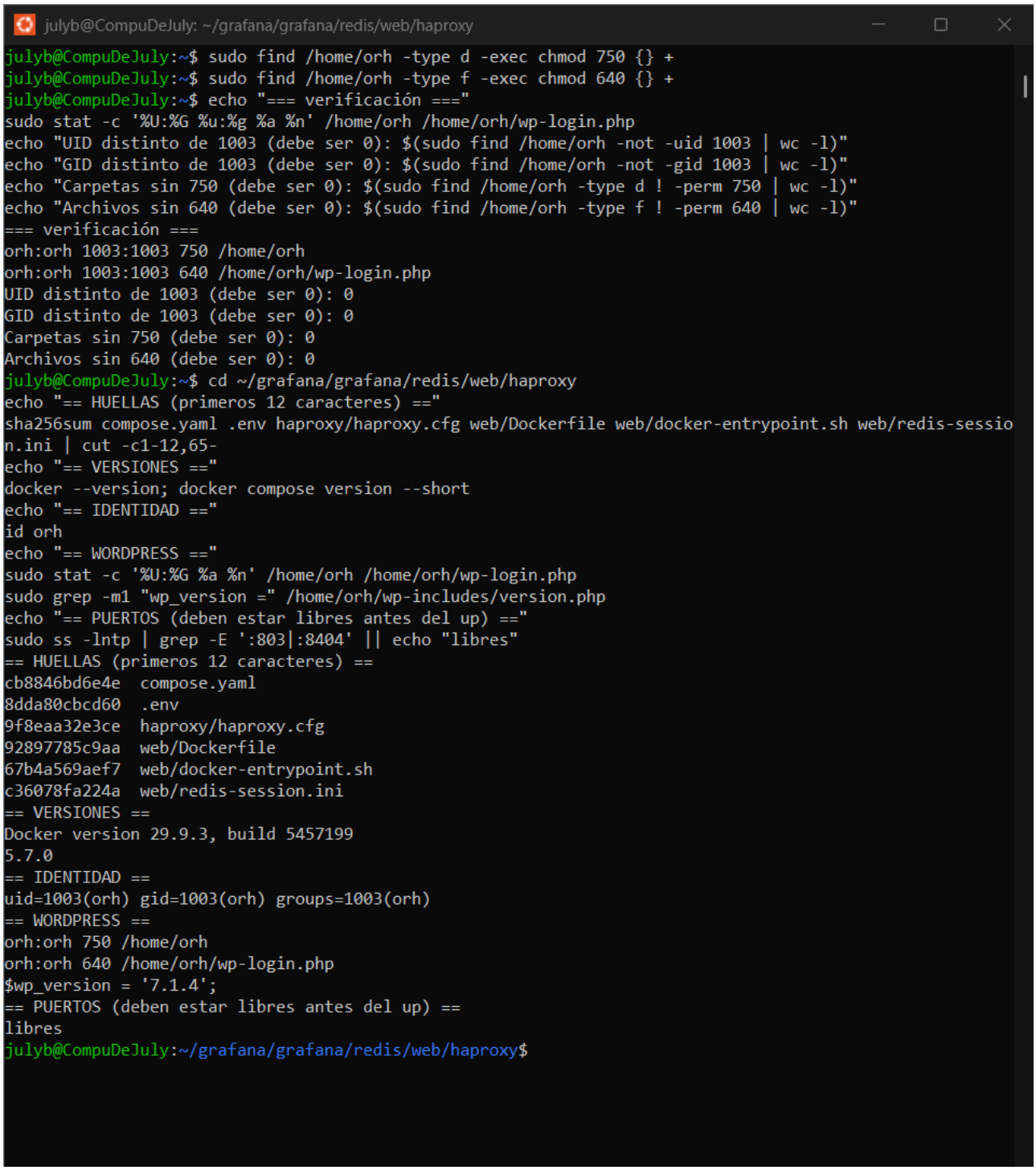
Error de "Permission denied" al intentar ejecutar el script setup-taller2.sh.



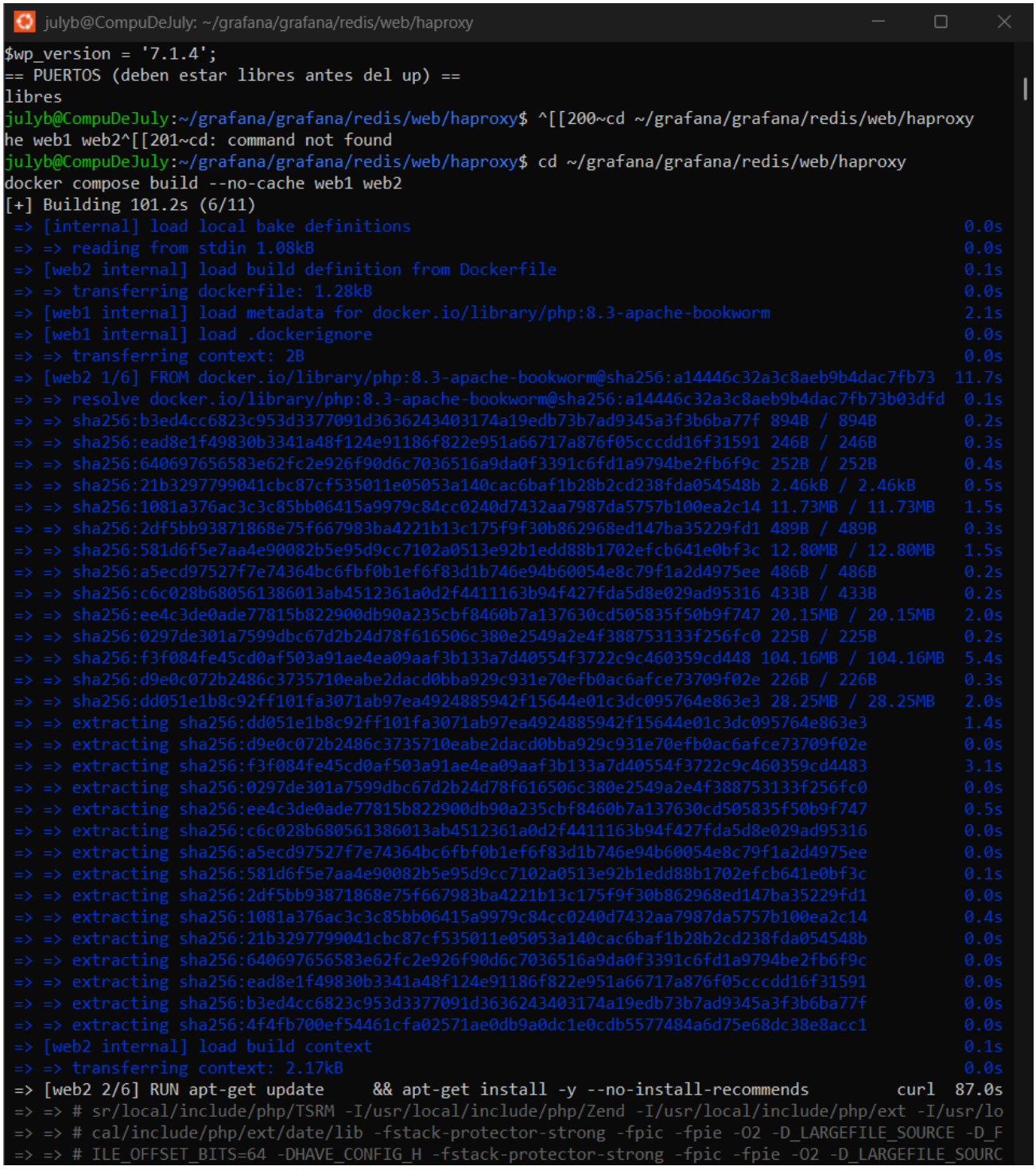
Corrección de permisos del script y ejecución exitosa (archivos del proyecto creados).



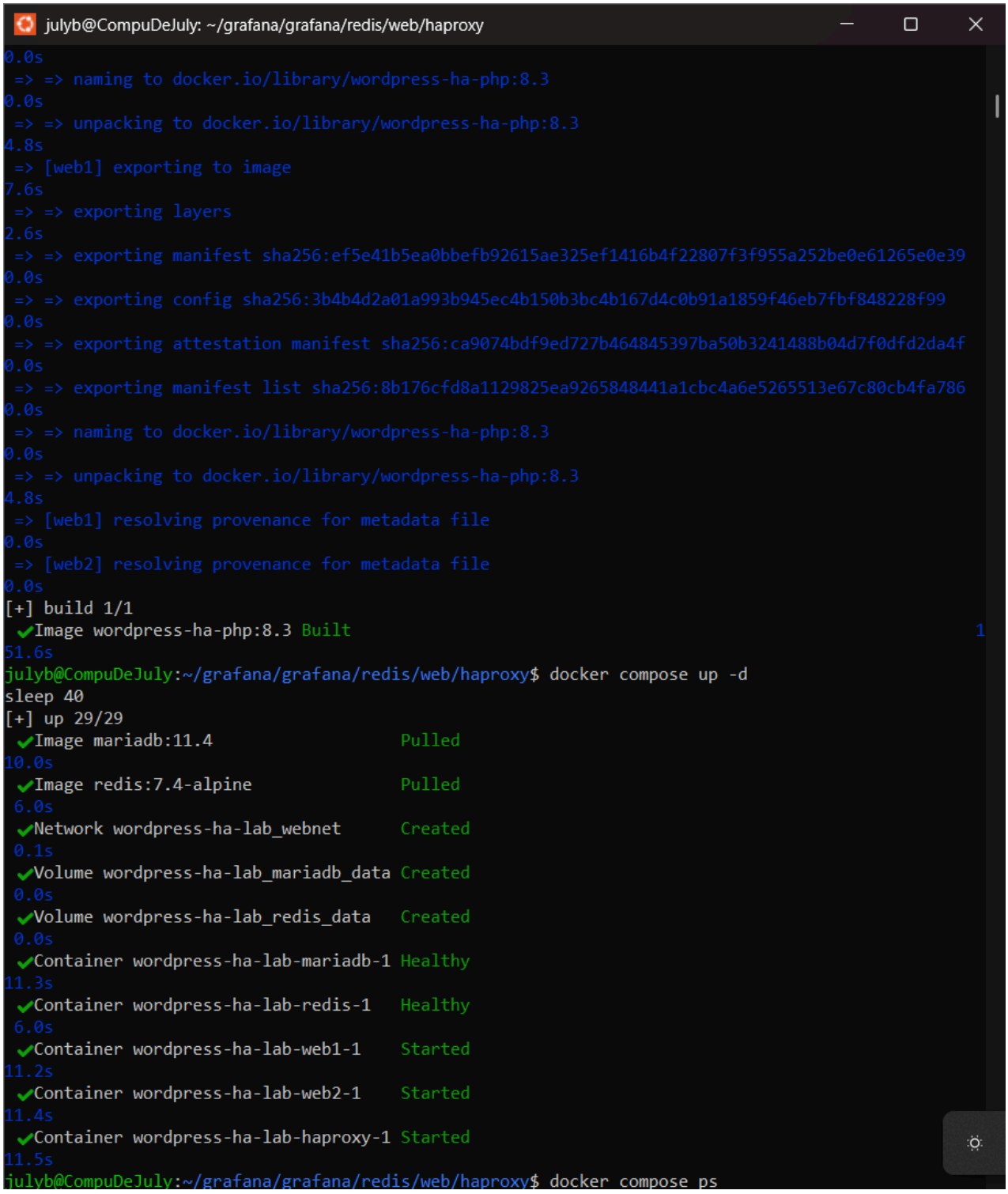
Verificación de que los archivos de WordPress pertenecen a UID/GID 1003 y que los puertos están libres.



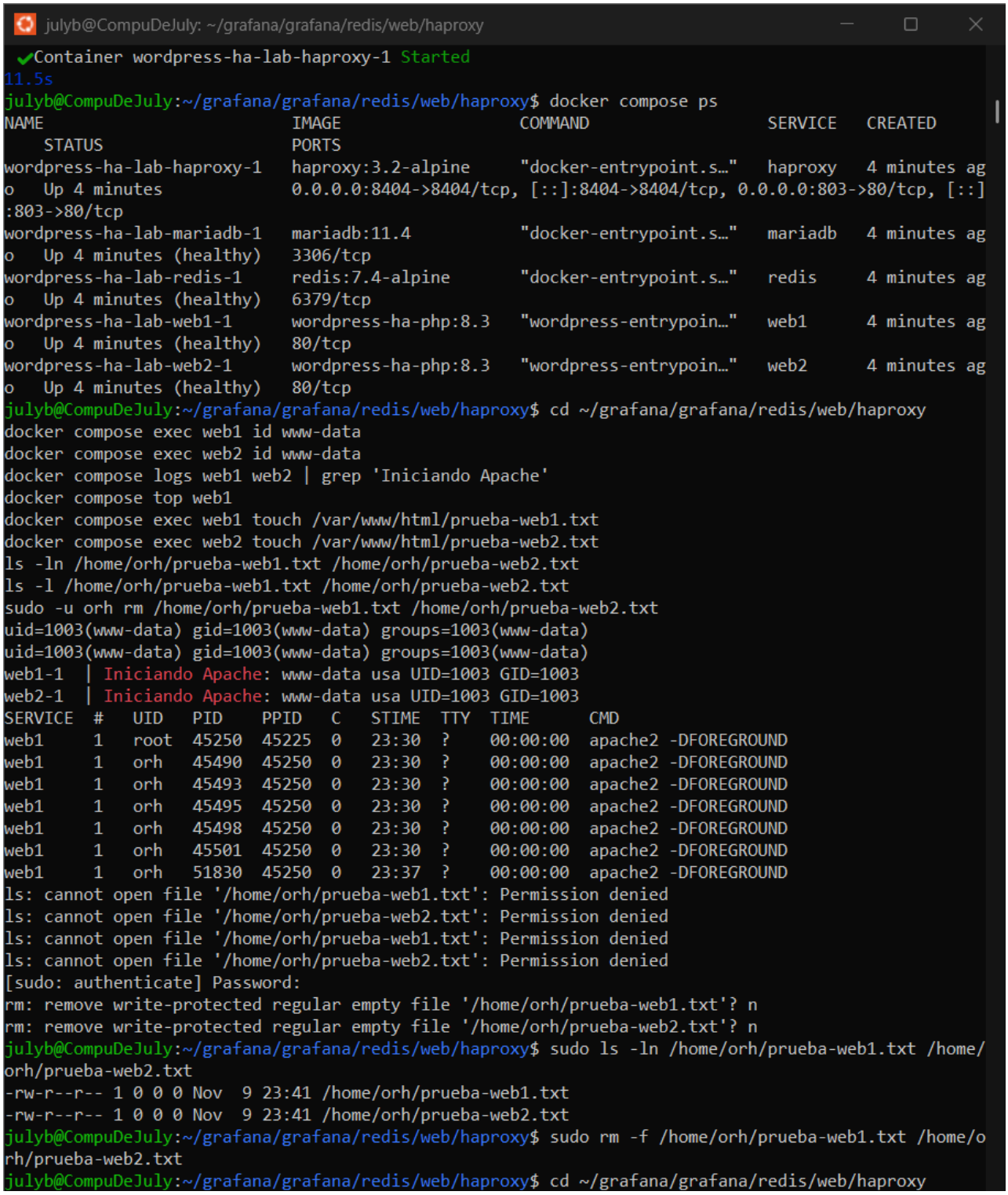
Comprobación de huellas de archivos (hashes), versiones de Docker y estado del WordPress (7.1.4).



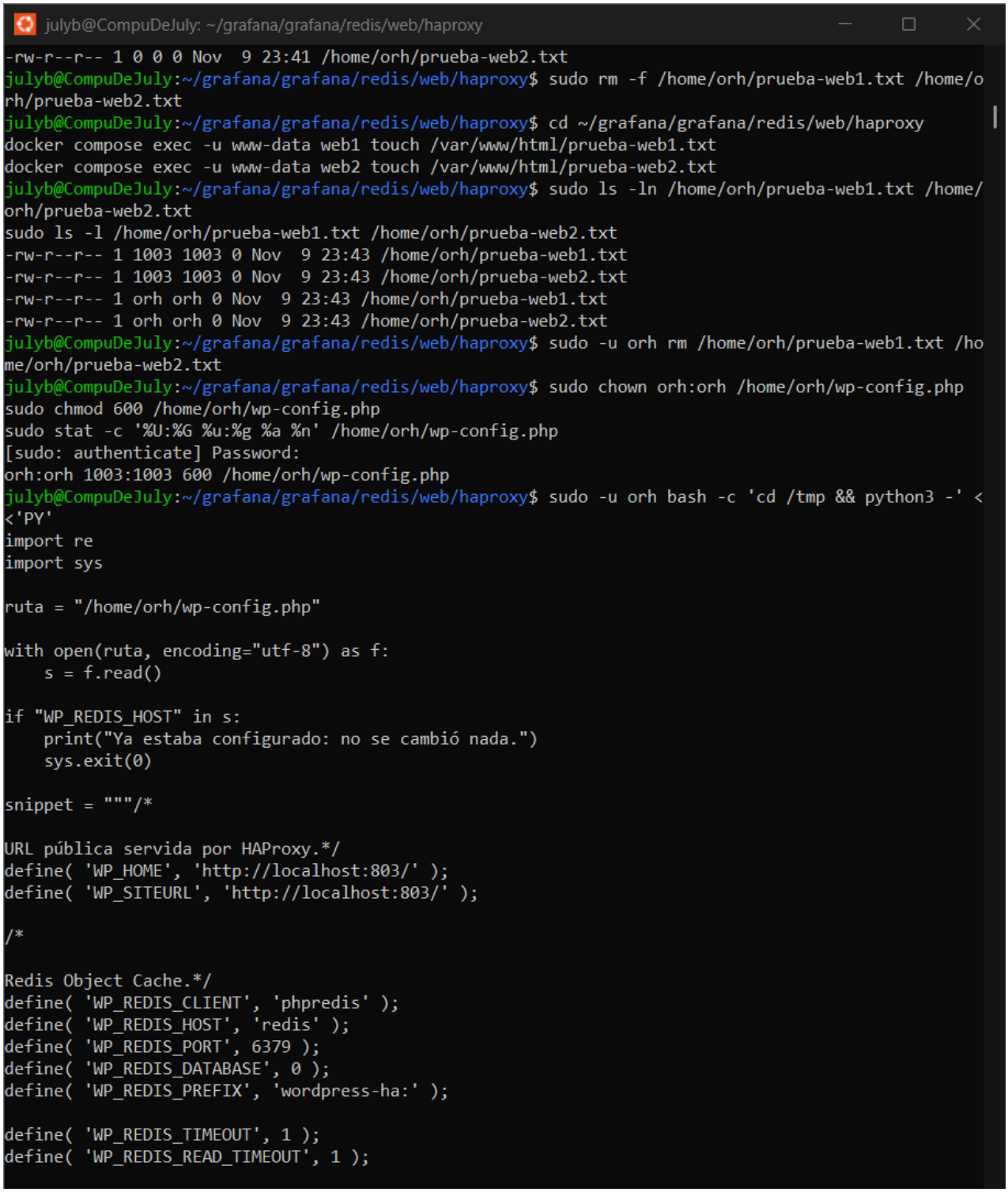
Construcción de las imágenes de web1 y web2 (docker compose build).



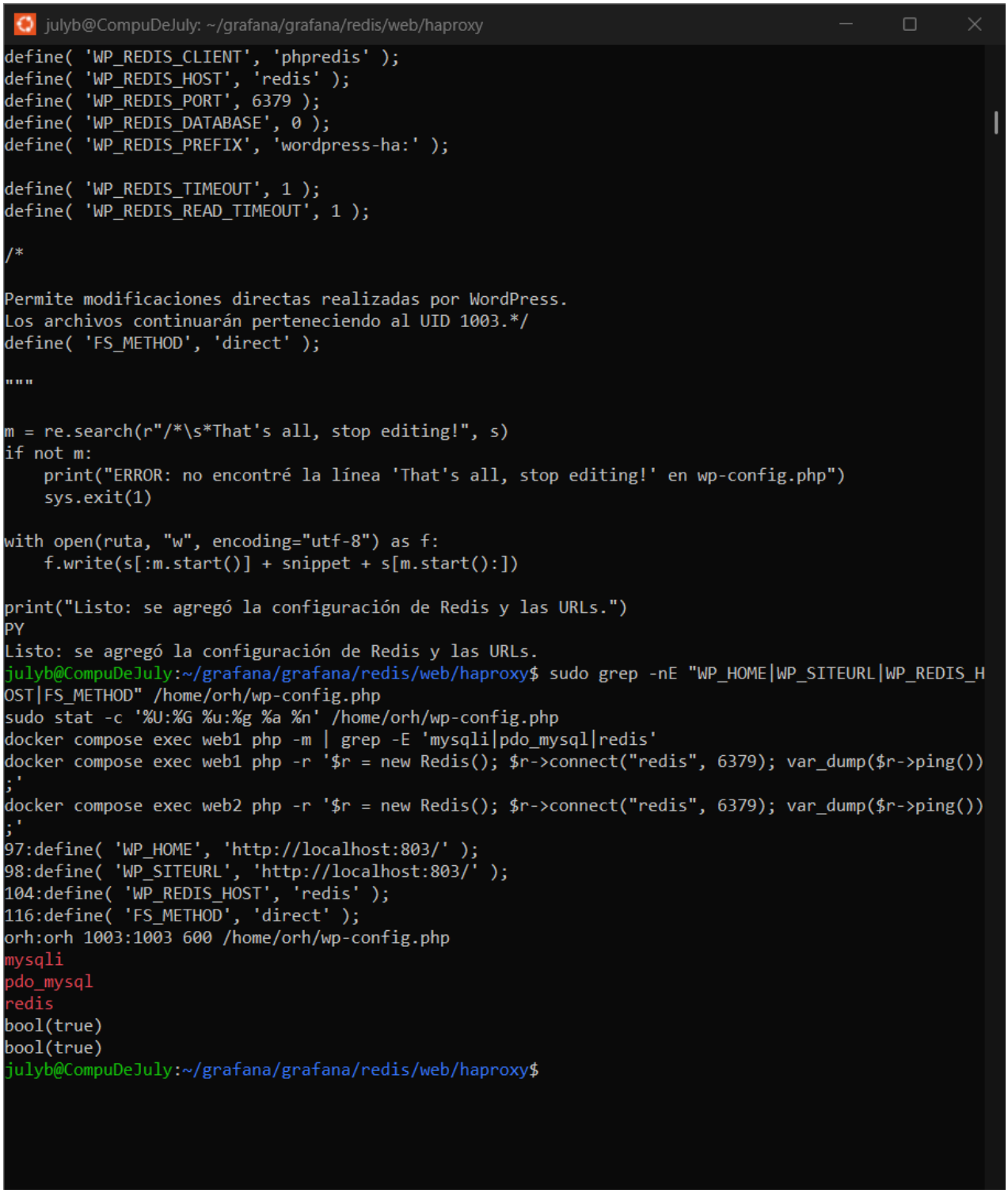
Salida del proceso de construcción de la imagen (descarga de capas base y exportación).



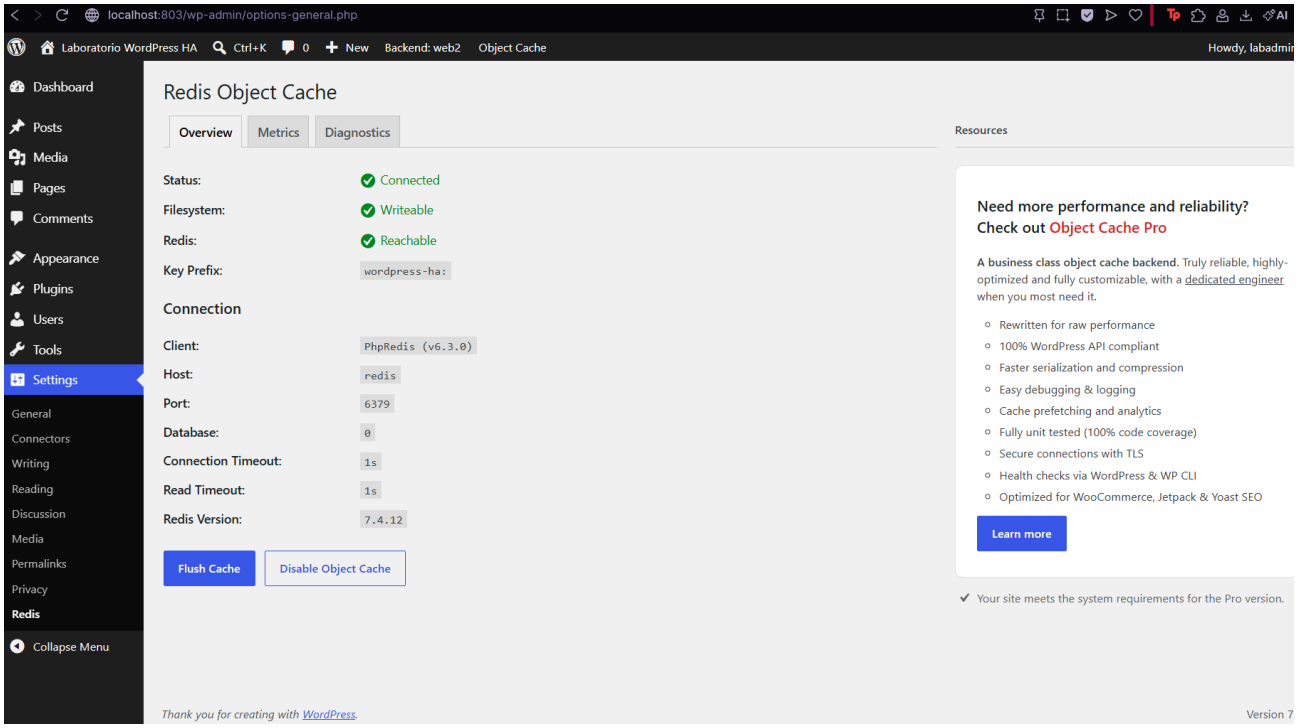
Ajuste de permisos del archivo wp-config.php (asignando UID 1003 y permisos 600) y edición de dicho archivo para insertar la configuración de Redis y las URLs públicas de WordPress.



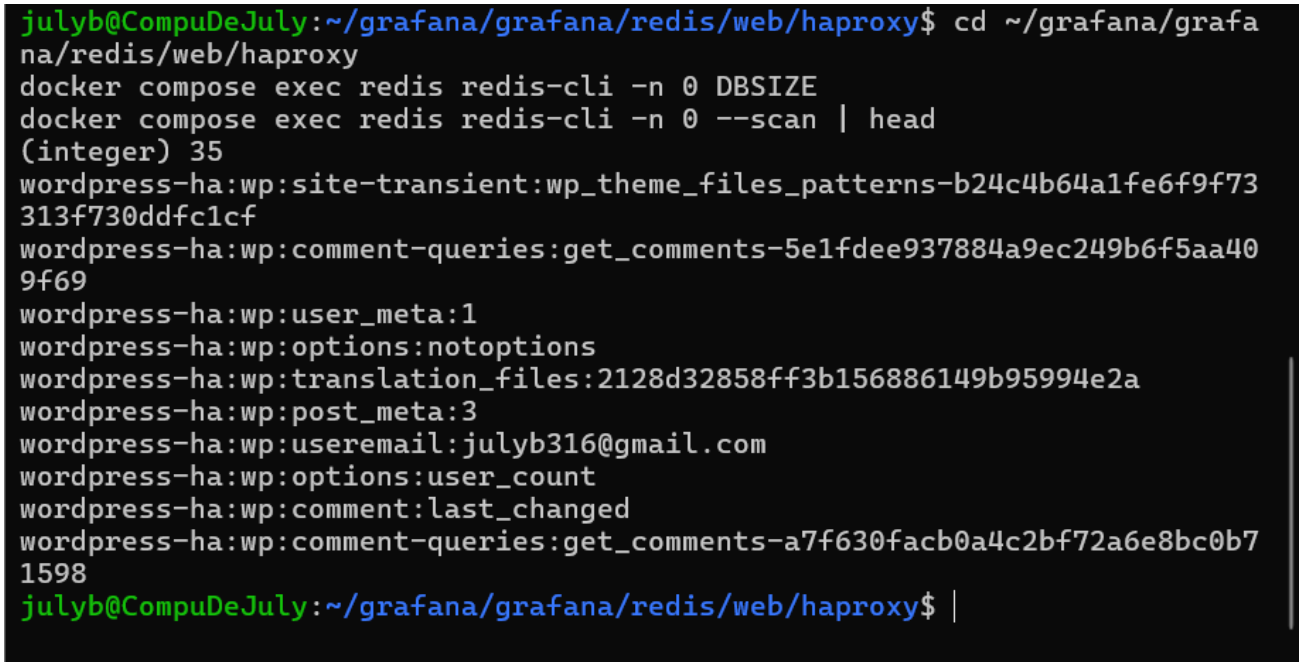
Verificación de las líneas añadidas en wp-config.php, comprobación de las extensiones PHP necesarias (mysql, pdo_mysql, redis) y prueba exitosa de conexión a Redis desde ambos contenedores web (web1 y web2), devolviendo bool(true).



Captura del panel de administración de WordPress mostrando el plugin "Redis Object Cache" en estado "Connected" (conectado), apuntando al host redis.



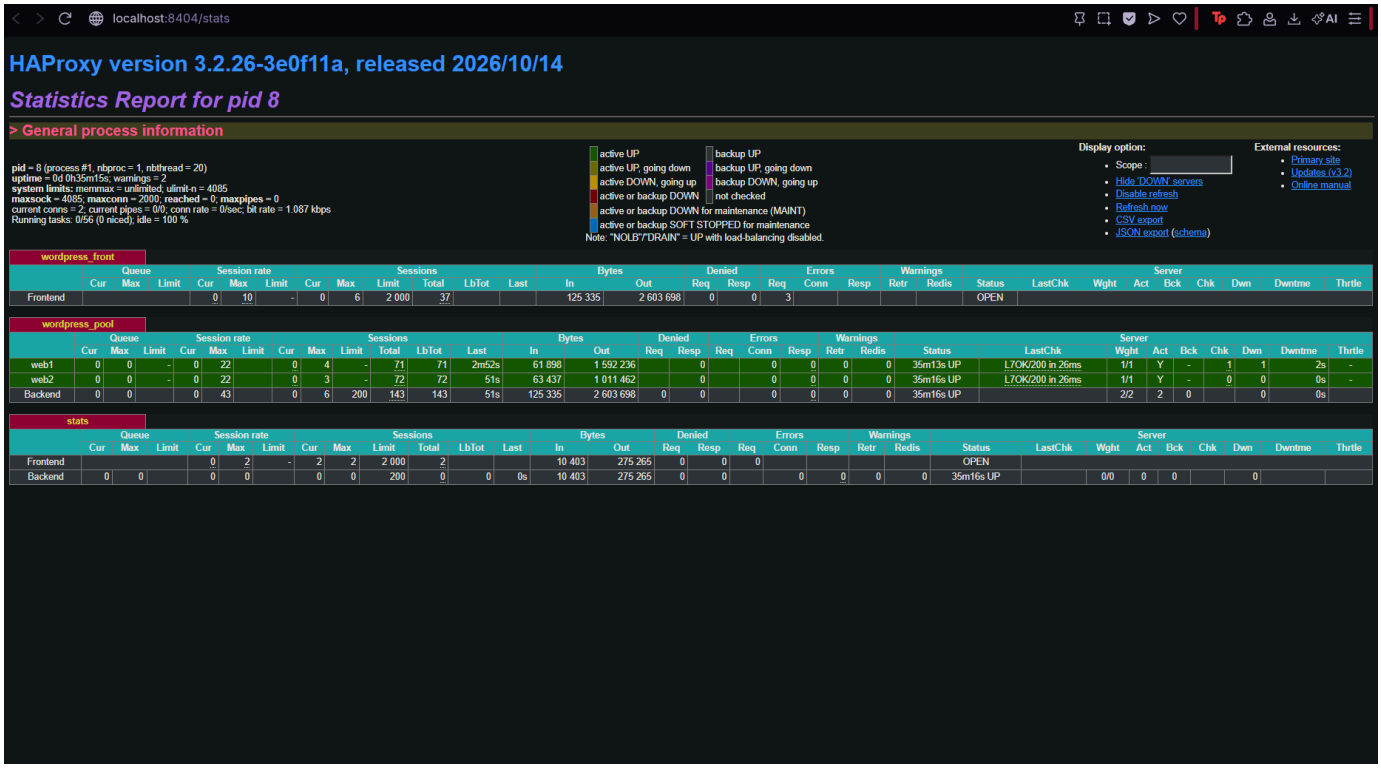
Captura de la terminal consultando la base de datos de Redis con redis-cli, confirmando que WordPress está guardando su caché de objetos (35 claves) con el prefijo wordpress-ha:.



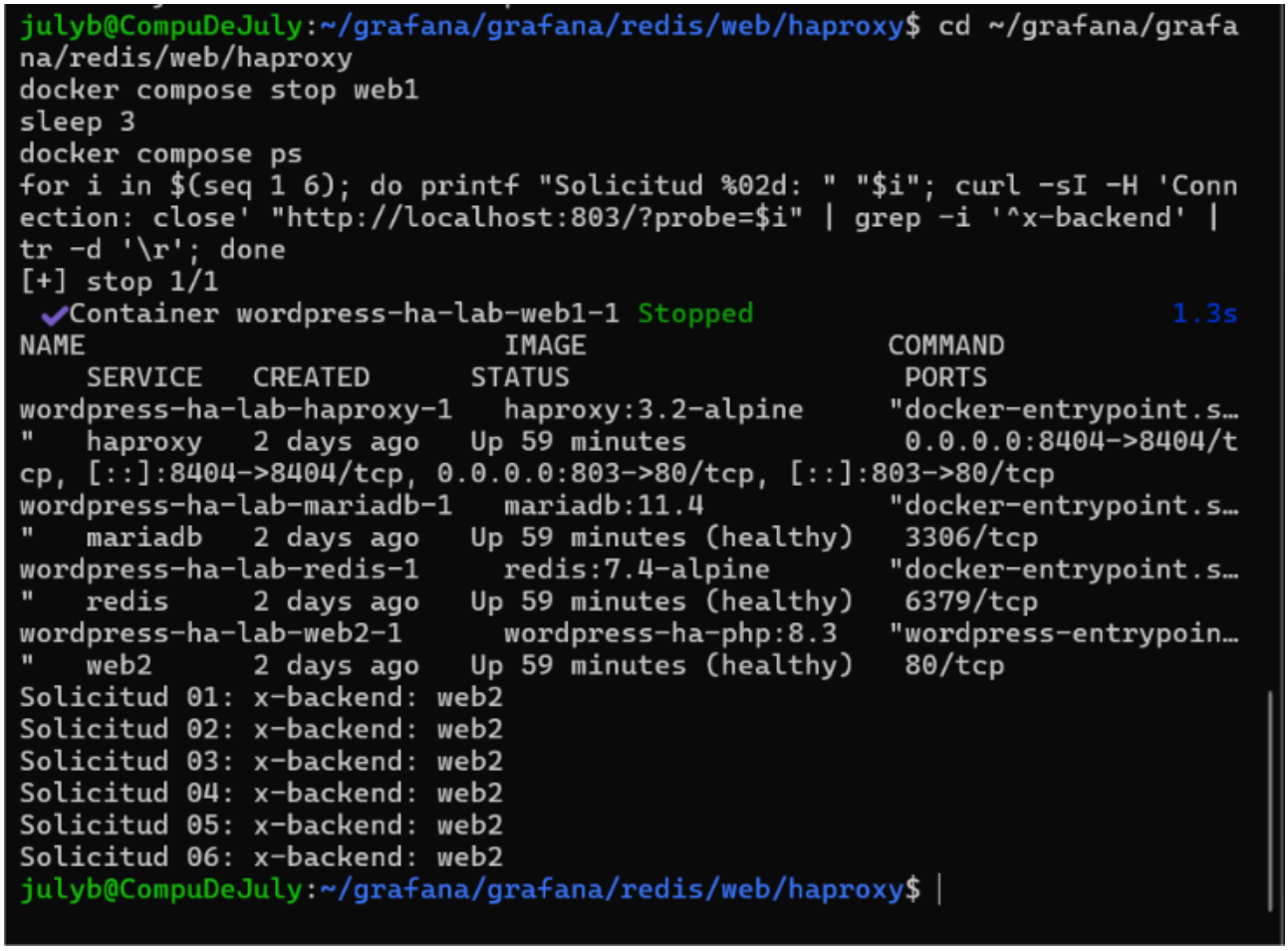
Prueba de balanceo (Round Robin) con curl: el tráfico alterna entre web1 y web2.

```
julyb@CompuDeJuly:~/grafana/grafana/redis/web/haproxy$ for i in $(seq 1 1
0); do printf "Solicitud %02d: " "$i"; curl -sI -H 'Connection: close' "h
ttp://localhost:803/?probe=$i" | grep -i '^x-backend' | tr -d '\r'; done
Solicitud 01: x-backend: web2
Solicitud 02: x-backend: web1
Solicitud 03: x-backend: web2
Solicitud 04: x-backend: web1
Solicitud 05: x-backend: web2
Solicitud 06: x-backend: web1
Solicitud 07: x-backend: web2
Solicitud 08: x-backend: web1
Solicitud 09: x-backend: web2
Solicitud 10: x-backend: web1
julyb@CompuDeJuly:~/grafana/grafana/redis/web/haproxy$
```

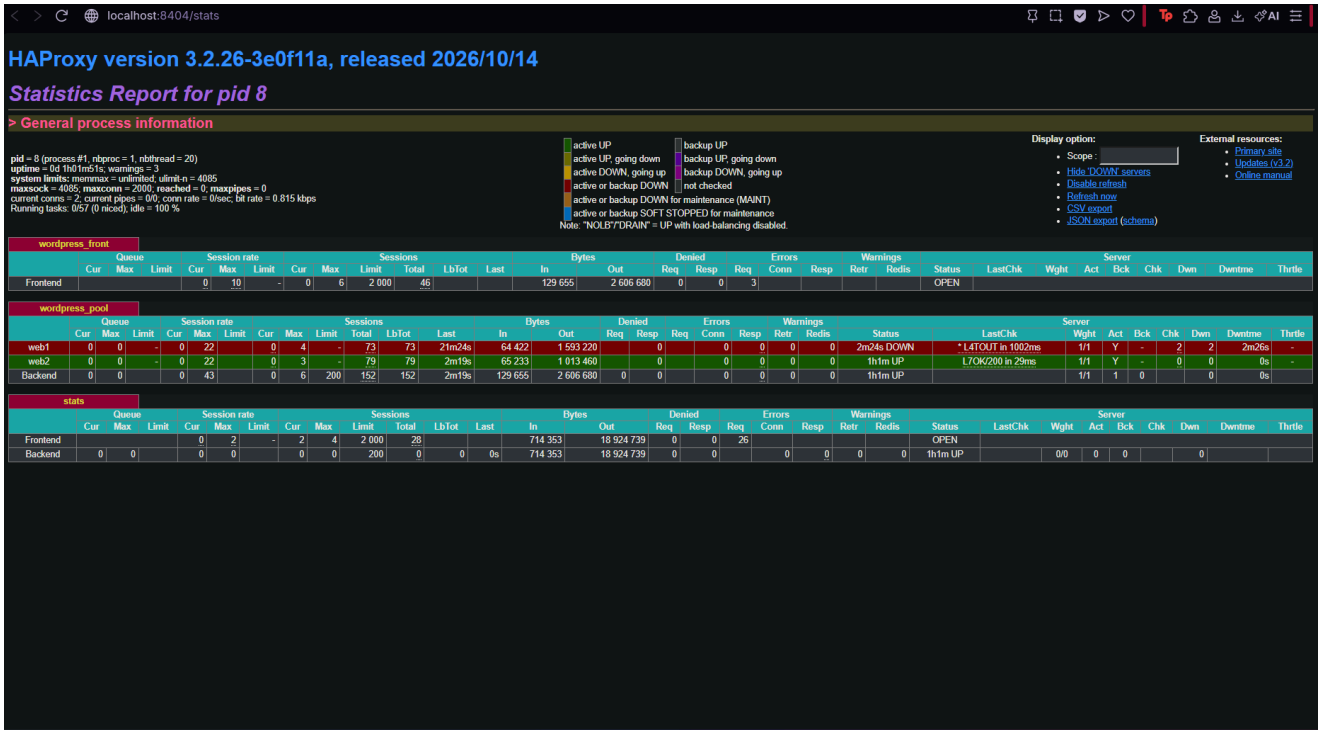
Panel de HAProxy mostrando ambos servidores en estado UP (verde)



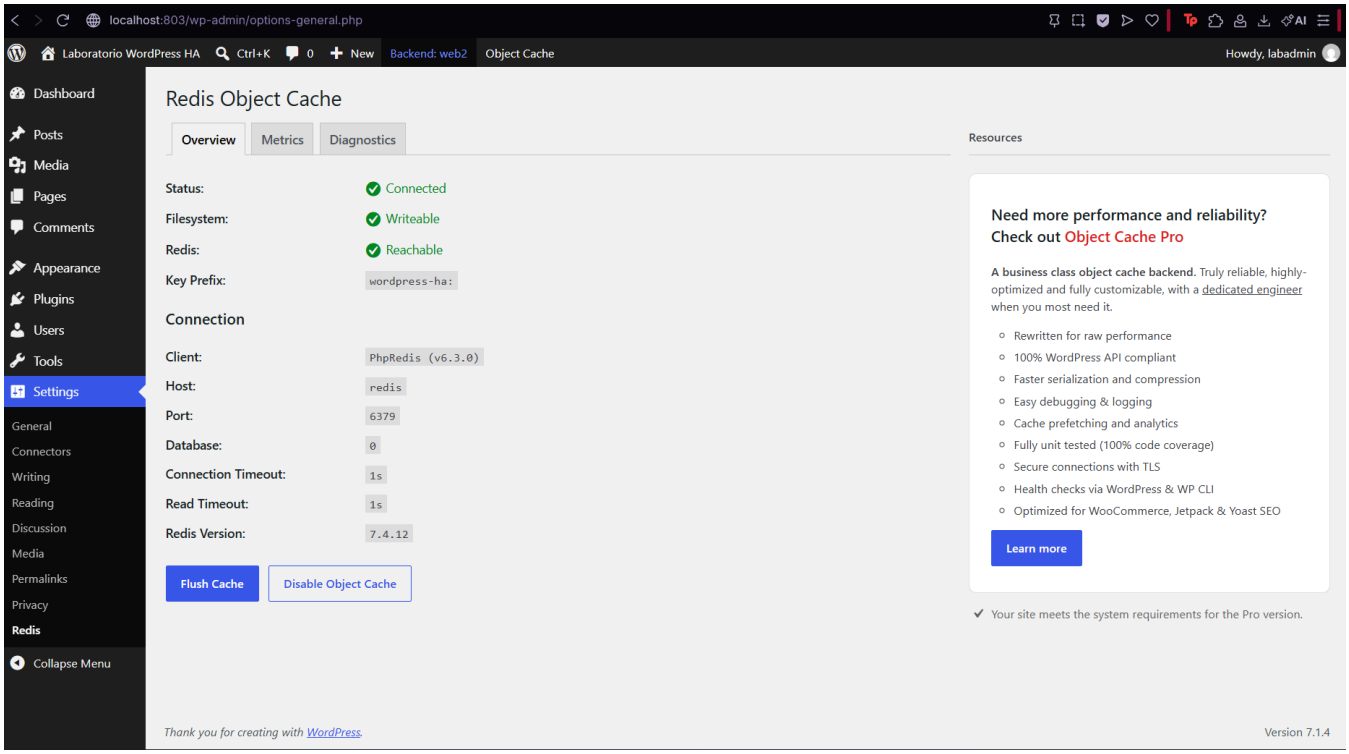
Prueba 1: Se detiene web1. HAProxy lo marca como DOWN y envía todo el tráfico a web2.



Panel de HAProxy mostrando web1 en rojo (DOWN) y web2 absorbiendo todo.



Se reinicia web1. Tras los chequeos exitosos, HAProxy vuelve a balancear el tráfico.



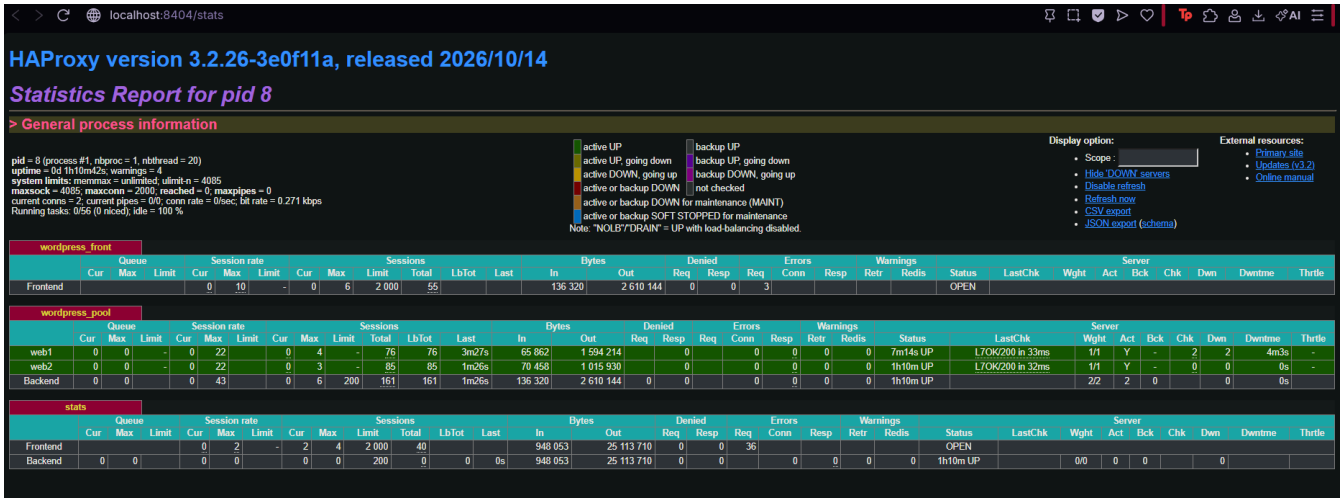
Panel de WordPress confirmando que Redis sigue funcionando.

```
julyb@CompuDeJuly:~/grafana/grafana/redis/web/haproxy$ cd ~/grafana/grafa
na/redis/web/haproxy
docker compose start web1
sleep 15
docker compose ps
[+] start 3/3
 ✔Container wordpress-ha-lab-mariadb-1 Healthy                    0.0s
 ✔Container wordpress-ha-lab-redis-1   Healthy                    0.0s
 ✔Container wordpress-ha-lab-web1-1    Started                    0.4s
NAME                          IMAGE                  COMMAND
    SERVICE   CREATED      STATUS                      PORTS
wordpress-ha-lab-haproxy-1   haproxy:3.2-alpine     "docker-entrypoint.s…
"   haproxy   2 days ago   Up About an hour            0.0.0.0:8404->840
4/tcp, [::]:8404->8404/tcp, 0.0.0.0:803->80/tcp, [::]:803->80/tcp
wordpress-ha-lab-mariadb-1   mariadb:11.4           "docker-entrypoint.s…
"   mariadb   2 days ago   Up About an hour (healthy)   3306/tcp
wordpress-ha-lab-redis-1     redis:7.4-alpine       "docker-entrypoint.s…
"   redis     2 days ago   Up About an hour (healthy)   6379/tcp
wordpress-ha-lab-web1-1      wordpress-ha-php:8.3   "wordpress-entrypoin…
"   web1      2 days ago   Up 15 seconds (healthy)      80/tcp
wordpress-ha-lab-web2-1      wordpress-ha-php:8.3   "wordpress-entrypoin…
"   web2      2 days ago   Up About an hour (healthy)   80/tcp
julyb@CompuDeJuly:~/grafana/grafana/redis/web/haproxy$
```

Prueba 2: Se detiene web2. HAProxy envía todo el tráfico a web1.

```
julyb@CompuDeJuly:~/grafana/grafana/redis/web/haproxy$ for i in $(seq 1 4
); do printf "Solicitud %02d: " "$i"; curl -sI -H 'Connection: close' "ht
tp://localhost:803/?probe=$i" | grep -i '^x-backend' | tr -d '\r'; done
Solicitud 01: x-backend: web1
Solicitud 02: x-backend: web2
Solicitud 03: x-backend: web1
Solicitud 04: x-backend: web2
julyb@CompuDeJuly:~/grafana/grafana/redis/web/haproxy$
```

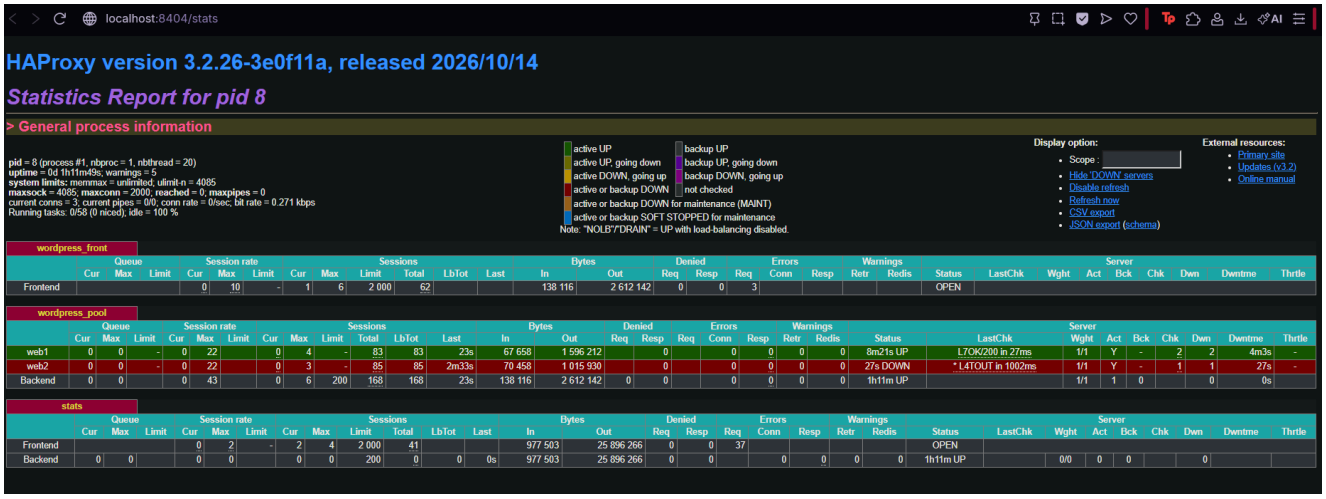
Panel de HAProxy mostrando web2 en rojo (DOWN) y web1 absorbiendo todo.



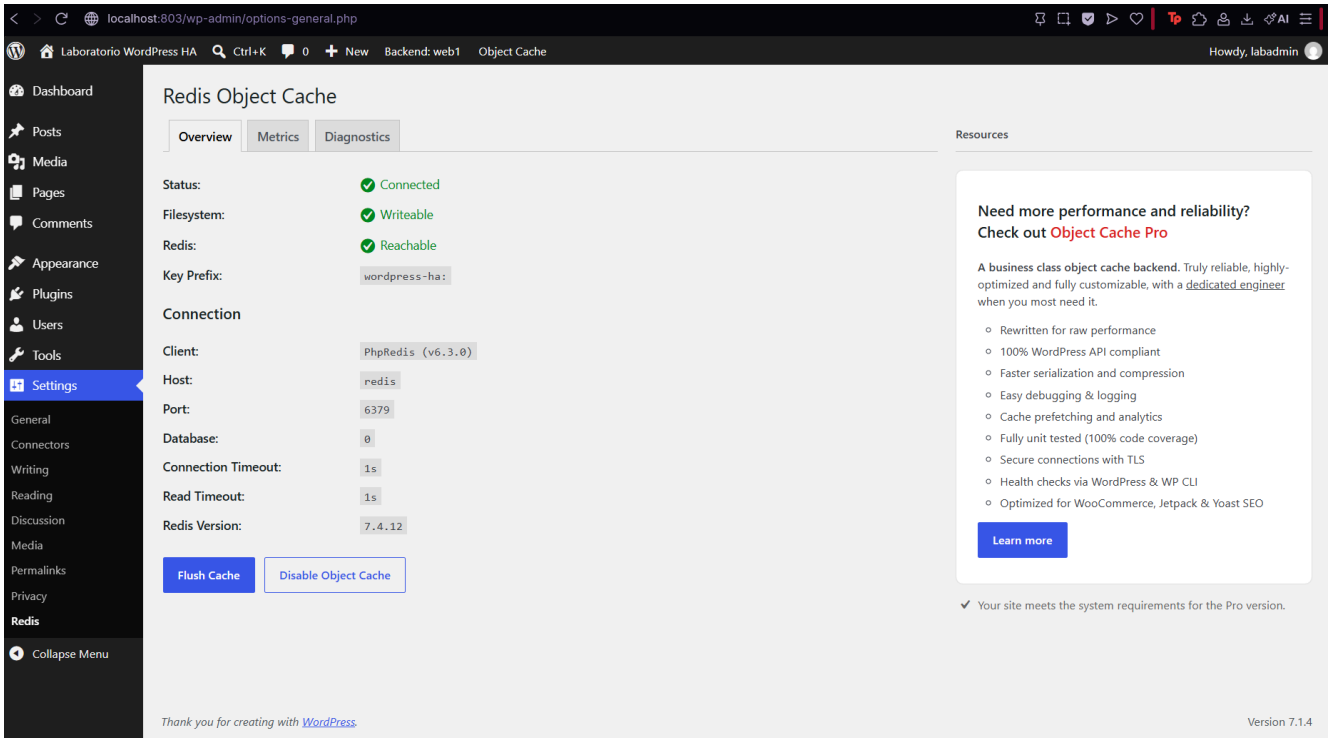
Se reinicia web2 y el tráfico vuelve a alternar.

```
julyb@CompuDeJuly:~/grafana/grafana/redis/web/haproxy$ cd ~/grafana/grafa
na/redis/web/haproxy
docker compose stop web2
sleep 3
docker compose ps
for i in $(seq 1 6); do printf "Solicitud %02d: " "$i"; curl -sI -H 'Conn
ection: close' "http://localhost:803/?probe=$i" | grep -i '^x-backend' |
tr -d '\r'; done
[+] stop 1/1
 ✔Container wordpress-ha-lab-web2-1 Stopped                          1.3s
NAME                          IMAGE                  COMMAND
    SERVICE   CREATED      STATUS                      PORTS
wordpress-ha-lab-haproxy-1    haproxy:3.2-alpine     "docker-entrypoint.s…
"   haproxy   2 days ago   Up About an hour            0.0.0.0:8404->840
4/tcp, [::]:8404->8404/tcp, 0.0.0.0:803->80/tcp, [::]:803->80/tcp
wordpress-ha-lab-mariadb-1    mariadb:11.4           "docker-entrypoint.s…
"   mariadb   2 days ago   Up About an hour (healthy)   3306/tcp
wordpress-ha-lab-redis-1      redis:7.4-alpine       "docker-entrypoint.s…
"   redis     2 days ago   Up About an hour (healthy)   6379/tcp
wordpress-ha-lab-web1-1       wordpress-ha-php:8.3   "wordpress-entrypoin…
"   web1      2 days ago   Up 8 minutes (healthy)       80/tcp
Solicitud 01: x-backend: web1
Solicitud 02: x-backend: web1
Solicitud 03: x-backend: web1
Solicitud 04: x-backend: web1
Solicitud 05: x-backend: web1
Solicitud 06: x-backend: web1
julyb@CompuDeJuly:~/grafana/grafana/redis/web/haproxy$
```

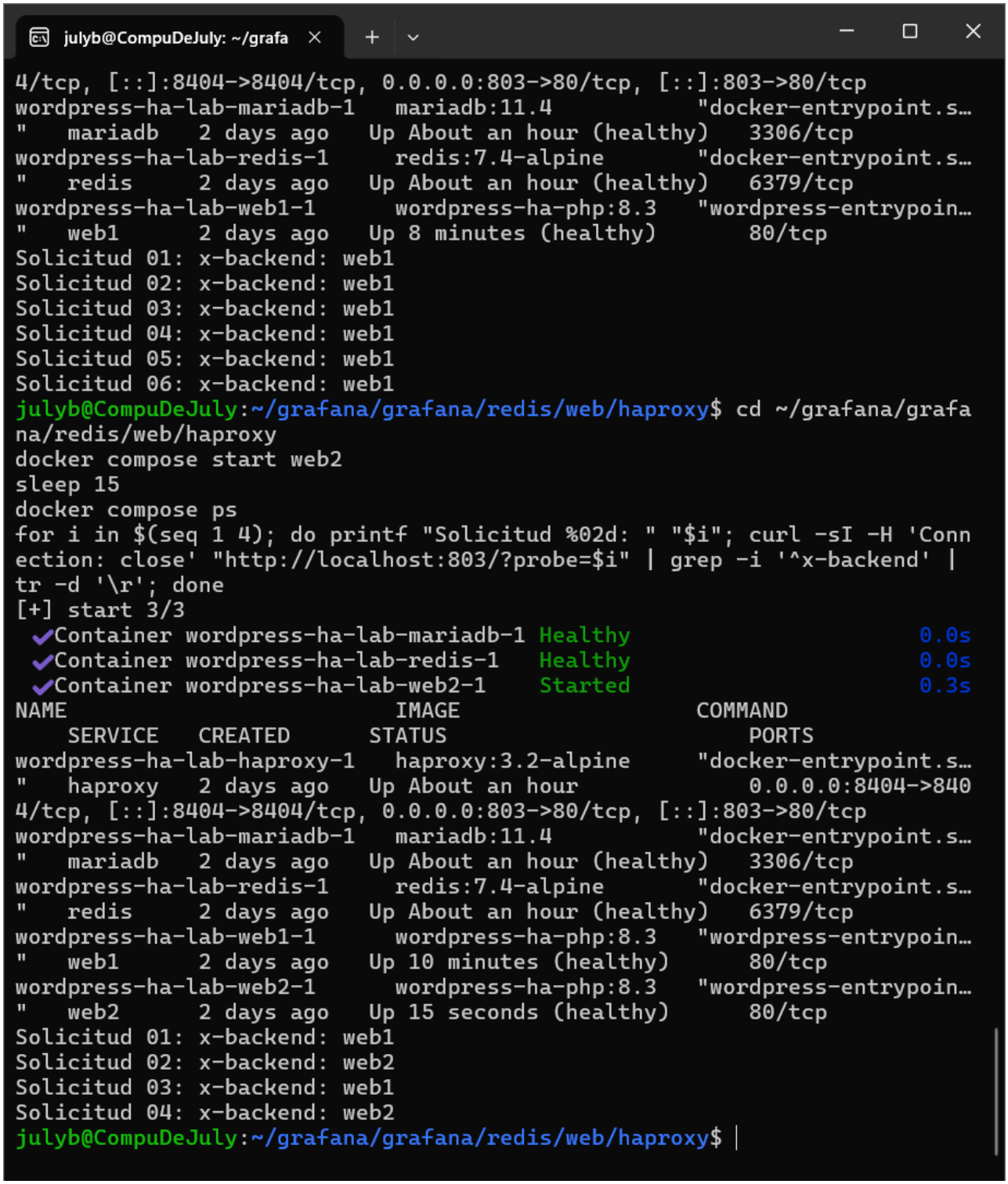
Panel de HAProxy mostrando ambos servidores UP (verde).



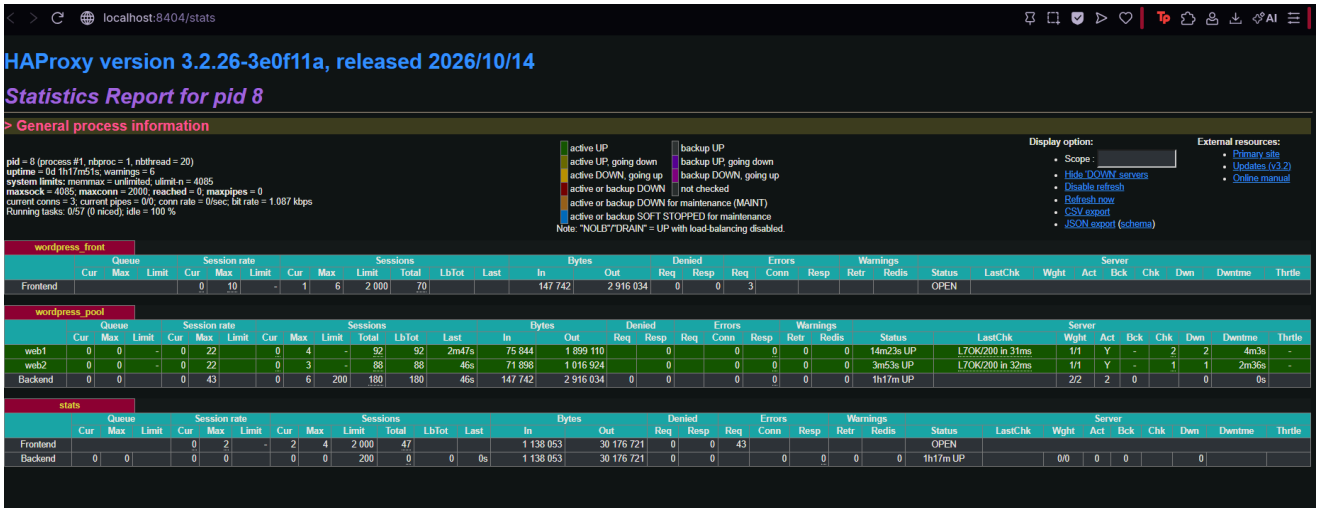
Panel de WordPress (Redis).



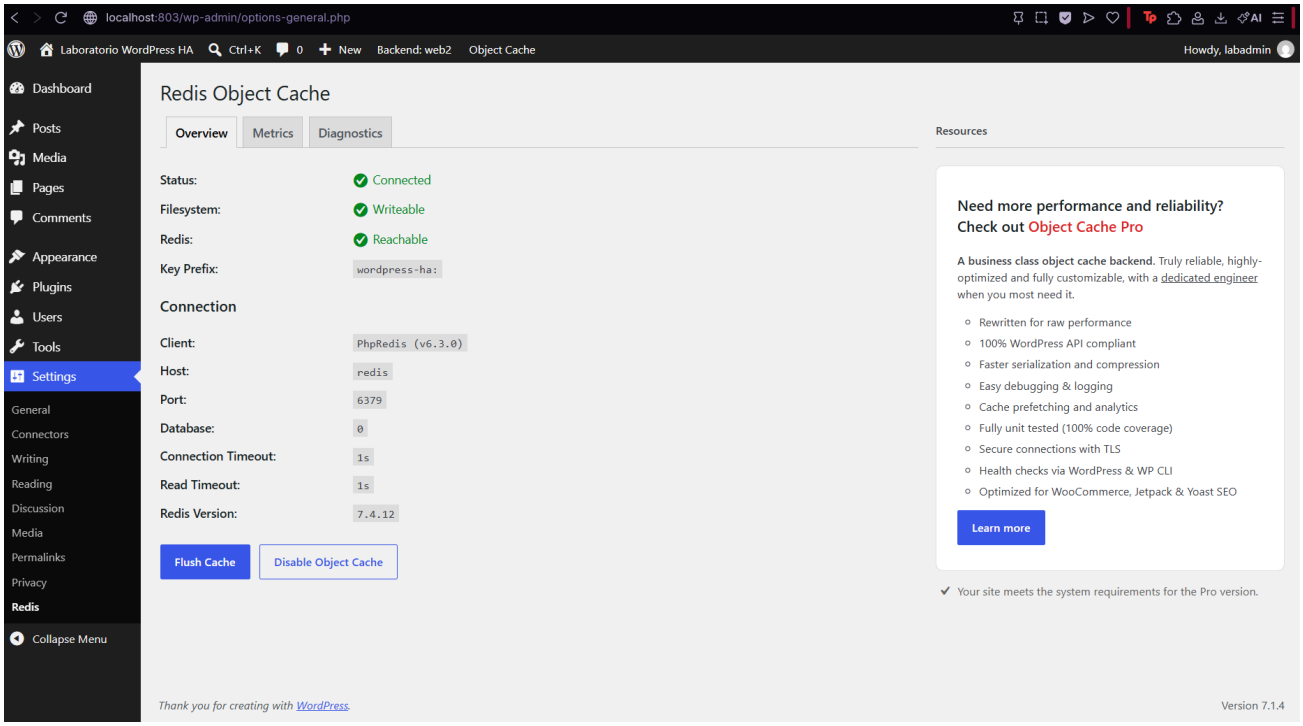
Terminal mostrando el balanceo tras reiniciar web2.



Panel de HAProxy con ambos servidores activos.



Panel de WordPress (Redis).



Se reinicia HAProxy y se comprueba que devuelve HTTP 200 y balancea correctamente.

```
julyb@CompuDeJuly:~/grafana/grafana/redis/web/haproxy$ docker compose res
tart haproxy
sleep 5
docker compose ps
for i in $(seq 1 4); do printf "Solicitud %02d: " "$i"; curl -sI -H 'Conn
ection: close' "http://localhost:803/?probe=$i" | grep -iE '^HTTP|^x-back
end' | tr -d '\r' | paste -sd' '; done
[+] restart 0/1
 ⠸ Container wordpress-ha-lab-haproxy-1 Restarting                  2.7s
NAME                          IMAGE                  COMMAND
    SERVICE   CREATED      STATUS                      PORTS
wordpress-ha-lab-haproxy-1    haproxy:3.2-alpine     "docker-entrypoint.s…
"   haproxy   2 days ago   Up 5 seconds                0.0.0.0:8404->840
4/tcp, [::]:8404->8404/tcp, 0.0.0.0:803->80/tcp, [::]:803->80/tcp
wordpress-ha-lab-mariadb-1    mariadb:11.4           "docker-entrypoint.s…
"   mariadb   2 days ago   Up About an hour (healthy)   3306/tcp
wordpress-ha-lab-redis-1      redis:7.4-alpine       "docker-entrypoint.s…
"   redis     2 days ago   Up About an hour (healthy)   6379/tcp
wordpress-ha-lab-web1-1       wordpress-ha-php:8.3   "wordpress-entrypoin…
"   web1      2 days ago   Up 19 minutes (healthy)      80/tcp
wordpress-ha-lab-web2-1       wordpress-ha-php:8.3   "wordpress-entrypoin…
"   web2      2 days ago   Up 9 minutes (healthy)       80/tcp
Solicitud 01: HTTP/1.1 200 OK x-backend: web1
Solicitud 02: HTTP/1.1 200 OK x-backend: web2
Solicitud 03: HTTP/1.1 200 OK x-backend: web1
Solicitud 04: HTTP/1.1 200 OK x-backend: web2
julyb@CompuDeJuly:~/grafana/grafana/redis/web/haproxy$
```

Preguntas de análisis

1. ¿Por qué es relevante que www-data tenga UID 1003 dentro de ambos contenedores?
Porque el WordPress no vive dentro de la imagen, sino que se monta desde el host con /home/orh:/var/www/html. En el host, /home/orh pertenece a orh con UID/GID 1003. Si dentro de los contenedores Apache/PHP corre como www-data con ese mismo UID/GID, puede leer y escribir los archivos sin pelearse con los permisos. Y como web1 y web2 comparten ese mismo volumen, ambos tienen que coincidir; si no, uno funciona y el otro empieza a dar errores raros.

2. ¿Qué ventaja ofrece ajustar el UID/GID en el entrypoint en lugar de incorporarlo como ARG de construcción?
La gran ventaja es que la imagen queda reutilizable. No la "casas" con un UID/GID fijo desde el build. Si mañana cambias de host o el usuario tiene otro UID, ajustas APP_UID y APP_GID en el .env y listo. Si lo metes como ARG, toca reconstruir la imagen para cada caso. Además, el entrypoint puede validar al arrancar si hay conflictos y ajustar www-data antes de lanzar Apache.

3. ¿Qué ocurriría si web1 utilizara UID 33 y web2 UID 1003?
Pues que tendrías un desastre intermitente. El volumen está pensado para 1003:1003, con permisos bastante cerrados: 750 en carpetas y 640 en archivos. web2 con UID 1003 probablemente trabajaría bien, pero web1 con UID 33 seguramente no podría atravesar /home/orh ni leer o escribir WordPress. HAProxy repartiría peticiones entre una réplica sana y otra con problemas de permisos, así que verías fallos aleatorios, errores al actualizar, subir archivos o guardar cambios.

4. ¿Por qué Redis Object Cache no es, por sí solo, el responsable de mantener el inicio de sesión nativo de WordPress?
Porque WordPress nativo no usa sesiones PHP para autenticar. Usa cookies wordpress_logged_in_*, y esas cookies se validan con las sales de wp-config.php y con los datos del usuario en MariaDB. Redis Object Cache acelera consultas y cachea objetos, pero no es la fuente de verdad del login. El archivo redis-session.ini solo afecta a plugins que llamen explícitamente a session_start().

5. ¿Qué datos deben ser idénticos o compartidos para que cualquier réplica valide la cookie de autenticación?
Tienen que coincidir varias cosas: la base de datos MariaDB con usuarios, opciones y metadatos; las claves y sales de wp-config.php (AUTH_KEY, SECURE_AUTH_KEY, LOGGED_IN_KEY, etc.); WP_HOME y WP_SITEURL, que aquí apuntan a http://localhost:803/; el prefijo de tablas; el dominio/puerto público; y el mismo código/versión de WordPress. Si además hay plugins que usan sesiones PHP, también necesitan un almacén compartido, en este caso Redis.

6. ¿Qué diferencia existe entre balanceo de carga y alta disponibilidad?
El balanceo de carga reparte peticiones entre varias réplicas para escalar y mejorar el

rendimiento. La alta disponibilidad va más allá: busca que el servicio siga vivo aunque falle un componente, usando redundancia, health checks y failover. Aquí HAProxy hace balanceo y permite tolerar la caída de una réplica web, pero eso no convierte todo el sistema en alta disponibilidad completa.

7. ¿Por qué esta solución no es completamente tolerante a fallos?
Porque aunque web1 y web2 sean redundantes, siguen quedando puntos únicos de fallo: un solo HAProxy, una sola MariaDB, un solo Redis, un solo host/WSL, un único volumen local /home/orh, un solo daemon Docker y una sola red. Si falla cualquiera de esos, el servicio puede caerse aunque las réplicas web estén sanas.

8. ¿Qué ocurre si falla MariaDB?
Que WordPress se queda sin la fuente de verdad. No puede leer ni escribir usuarios, posts, opciones, metadatos, validaciones de login, etc. Las dos réplicas web fallan funcionalmente y HAProxy no puede arreglarlo porque no hay otra base de datos a la que mandar tráfico. Es un punto único de fallo claro.

9. ¿Qué ocurre si falla Redis? Distinga entre caché de objetos y sesiones PHP de plugins.
Si cae Redis como caché de objetos, WordPress no pierde los datos reales porque eso está en MariaDB. Puede degradarse el rendimiento, perder caché o dar errores según cómo esté configurado el plugin Object Cache, pero la fuente de verdad sigue en la base de datos. Si cae Redis como almacén de sesiones PHP de plugins, entonces los plugins que usen session_start() pierden o no pueden recuperar sus sesiones. Ojo: el login nativo de WordPress no depende de eso.

10. ¿Qué limitación presenta /home/orh si las réplicas se trasladan a hosts distintos?
Que es un bind mount local del host. En otro host ese directorio no existe o no tiene los mismos archivos. Para réplicas en hosts distintos necesitarías almacenamiento compartido tipo NFS, CephFS, EFS, GlusterFS o un volumen RWX; o bien meter el código en la imagen y externalizar los uploads. También tendrías que asegurarte de que el UID/GID 1003 exista o se mapee correctamente en cada nodo.

11. ¿Qué función cumplen fall 1 y rise 2 en HAProxy?
fall 1 significa que con un solo chequeo fallido HAProxy marca el servidor como DOWN. rise 2 significa que necesita dos chequeos exitosos seguidos para volver a marcarlo como UP. Así evitas mandar tráfico a una réplica caída y también evitas reincorporarla demasiado pronto mientras todavía está arrancando.

12. ¿Por qué no se expone MariaDB ni Redis mediante ports?
Por seguridad y porque no hace falta. Solo web1 y web2 necesitan hablar con MariaDB y Redis dentro de la red webnet. No hay razón para publicarlos en el host o en Internet. Solo HAProxy expone 803 y 8404. Exponer la base de datos o la caché aumentaría la superficie de ataque sin darte ninguna ventaja.

13. ¿Qué riesgo representa que un usuario pertenezca al grupo docker?
Un usuario en el grupo docker puede ejecutar contenedores con montajes del host, privilegios elevados o incluso --privileged. En la práctica, eso permite escalar a root en la máquina anfitriona. Por eso se dice que pertenecer a docker es casi equivalente a tener control root sobre el host.

14. ¿Por qué una solicitud que estaba en ejecución podría fallar aunque exista otra réplica?
Porque HAProxy ya la asignó a un backend concreto. Si ese backend cae a mitad de la petición, la conexión se corta. HAProxy no migra una petición en curso a otra réplica, y por defecto tampoco reintenta cualquier método o petición ya enviada. El cliente tendría que volver a intentarlo.

15. ¿Qué componentes deberían duplicarse para eliminar los puntos únicos de fallo?
Habría que redundar HAProxy con una VIP o un balanceador externo, MariaDB con replicación/Galera y failover, Redis con Sentinel/Cluster o una réplica gestionada, el almacenamiento compartido, los hosts/nodos y el plano de orquestación. También conviene externalizar o replicar uploads y las sesiones de plugins. Mientras todo eso viva en un solo host y con una sola BD/caché, no puedes hablar de alta disponibilidad completa.

16. ¿Cómo cambiaría este laboratorio al implementarlo en Docker Swarm o Kubernetes?
Cambiaría bastante la forma de gestionar volúmenes, secretos y réplicas. En Kubernetes usarías PVC, idealmente RWX para uploads; Secrets/ConfigMaps para .env y wp-config.php; Deployments para las réplicas web; un Ingress o Service en lugar de HAProxy manual; StatefulSets o servicios gestionados para MariaDB/Redis; y readinessProbe/livenessProbe junto con securityContext/fsGroup para UID/GID. En Swarm serían servicios con réplicas, routing mesh, secrets/configs y volúmenes compartidos. La idea es la misma, pero el orquestador se encarga de gran parte del failover y la programación.